

학습 내용

3부. 데이터 분석 라이브러리 활용



10장. N차원 배열 다루기



11장. 데이터프레임과 시리즈



12장. 데이터 시각화

- 1. 시각화 개요
- 2. Matplotlib
- 3. Seaborn



13장. 웹 데이터 수집

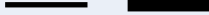
1절. 시각화 개요

```
# 시작전 설정
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format='retina'
#한글설정
plt.rc('font', family='Malgun Gothic')
#plt.rc('font', family='AppleGothic') #mac
plt.rc('axes', unicode_minus=False)
# 경고 메시지 안보이게
import warnings
warnings.filterwarnings(action='ignore')
```

1.1. 시각화를 통한 정보 전달

1절. 시각화 개요

- 시각화: 원본 데이터 또는 분석된 결과 데이터를 그래프로 표현하는 것

분류	정보 전달 수단	예시
점	크기	
	색/명도	
	모양	
선	굵기	
	색/명도	
평면	위치	
	면적	
	색/명도	
	질감	

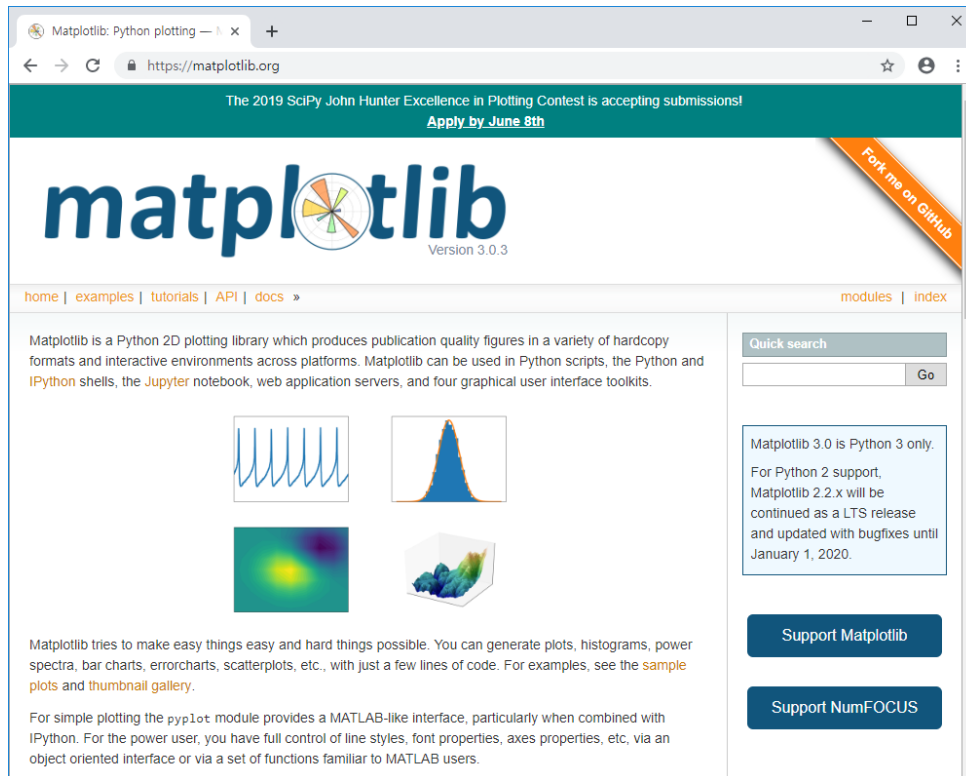


1.2. 시각화 라이브러리

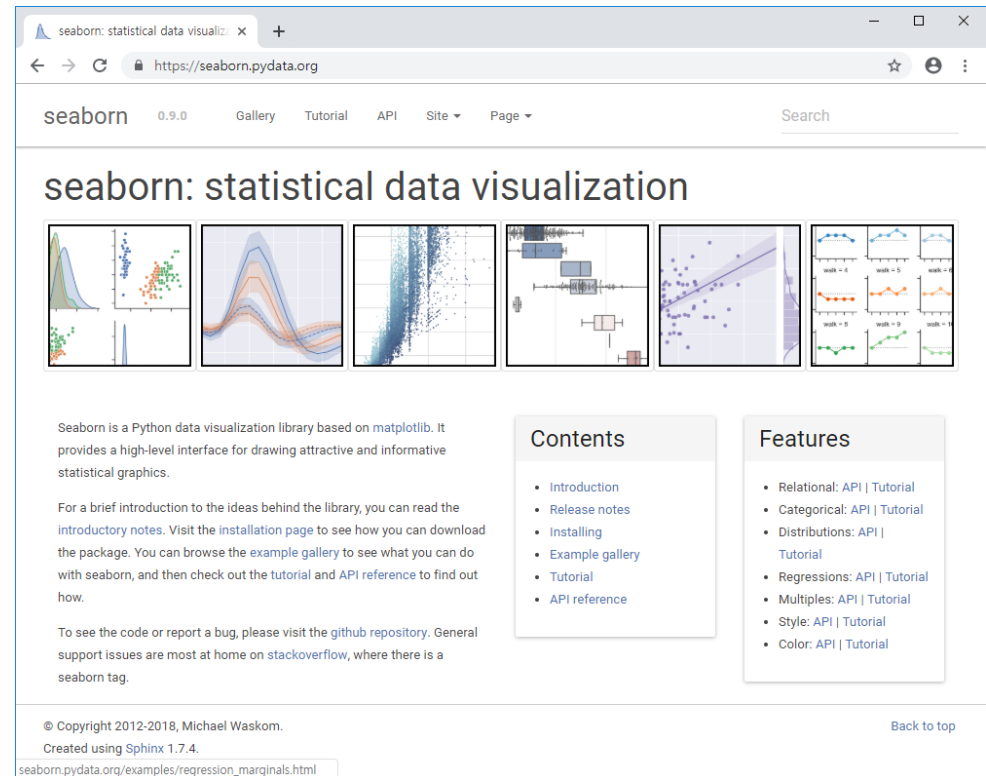
1절. 시각화 개요

- matplotlib, seaborn, plotnine, folium, poly.ly, pyecharts

<http://matplotlib.org>

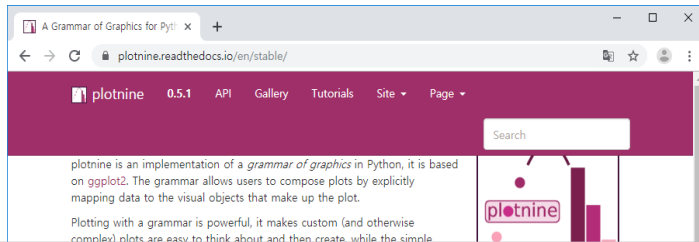


<https://seaborn.pydata.org>



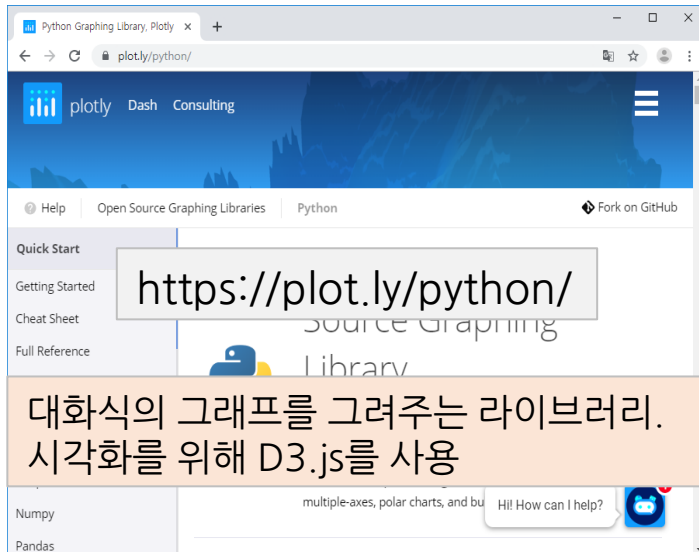
1.2. 시각화 라이브러리

1절. 시각화 개요



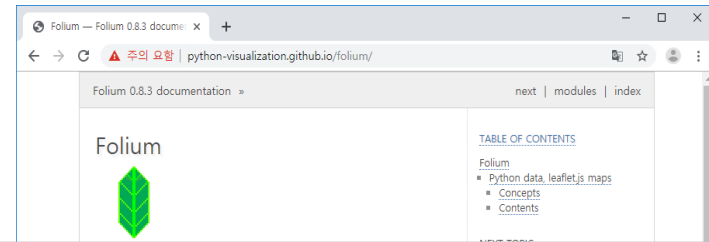
<https://plotnine.readthedocs.io/en/stable/>

ggplot2에 기반 한 그래프를 그려주는 라이브러리



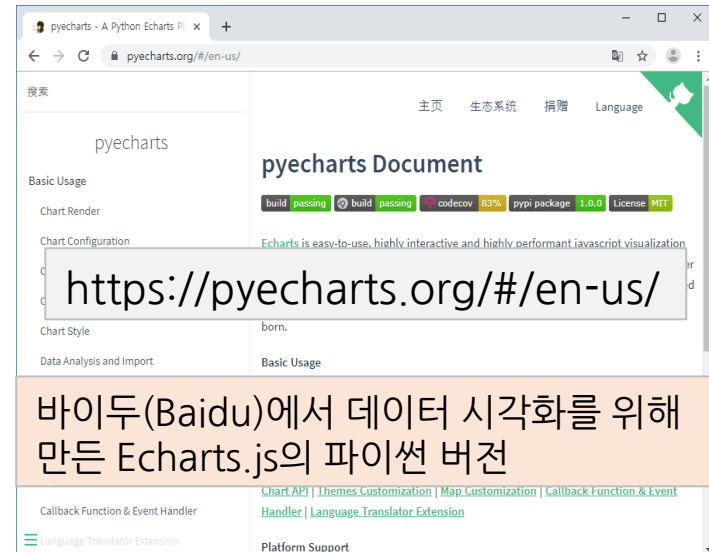
<https://plot.ly/python/>

대화식의 그래프를 그려주는 라이브러리. 시각화를 위해 D3.js를 사용



<http://python-visualization.github.io/folium/>

지도 데이터(Open Street Map)에 leaflet.js를 이용해 위치정보를 시각화하는 라이브러리



<https://pyecharts.org/#/en-us/>

바이두(Baidu)에서 데이터 시각화를 위해 만든 Echarts.js의 파이썬 버전

2절. Matplotlib

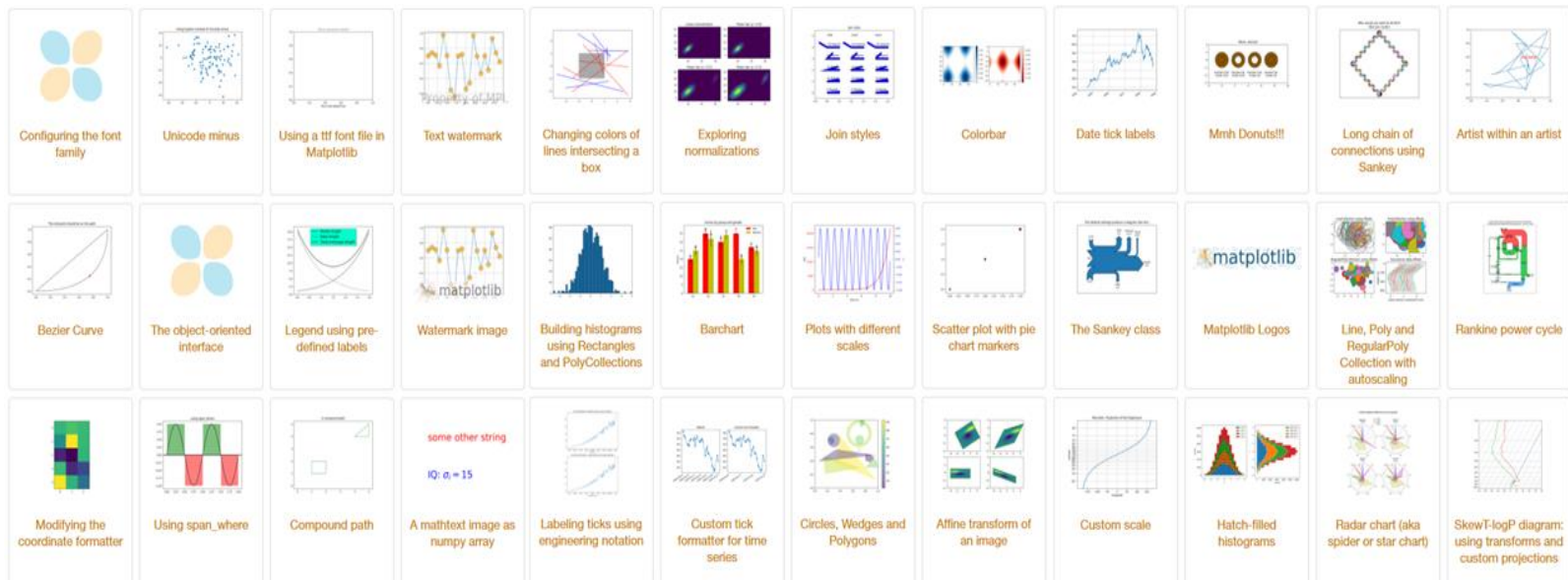
<https://pypi.org/project/matplotlib/>

https://matplotlib.org/stable/api/pyplot_summary.html

2절. Matplotlib

2절. Matplotlib

- Python 2D 플로팅 라이브러리
- 플랫폼에 독립적인 대화형 환경을 제공하며 고품질의 그림을 생성할 수 있게 해줌
- Python 스크립트, Python 및 IPython 셸, jupyter 노트북, 웹 응용 프로그램 서버에서 사용



2.1. 패키지 импорт 및 기본 설정

2절. Matplotlib

- Matplotlib로 그래프를 그리기 위한 라이브러리 import
- `%matplotlib inline`
 - notebook을 실행한 브라우저에서 바로 그림을 볼 수 있게 해 줌
- `%config InlineBackend.figure_format = 'retina'`
 - 그래프를 더 높은 해상도로 그려줌

```
1 import matplotlib
2 import matplotlib.pyplot as plt
```

```
1 %matplotlib inline
2 %config InlineBackend.figure_format = 'retina'
3 print("Matplotlib 버전:", matplotlib.__version__)
```

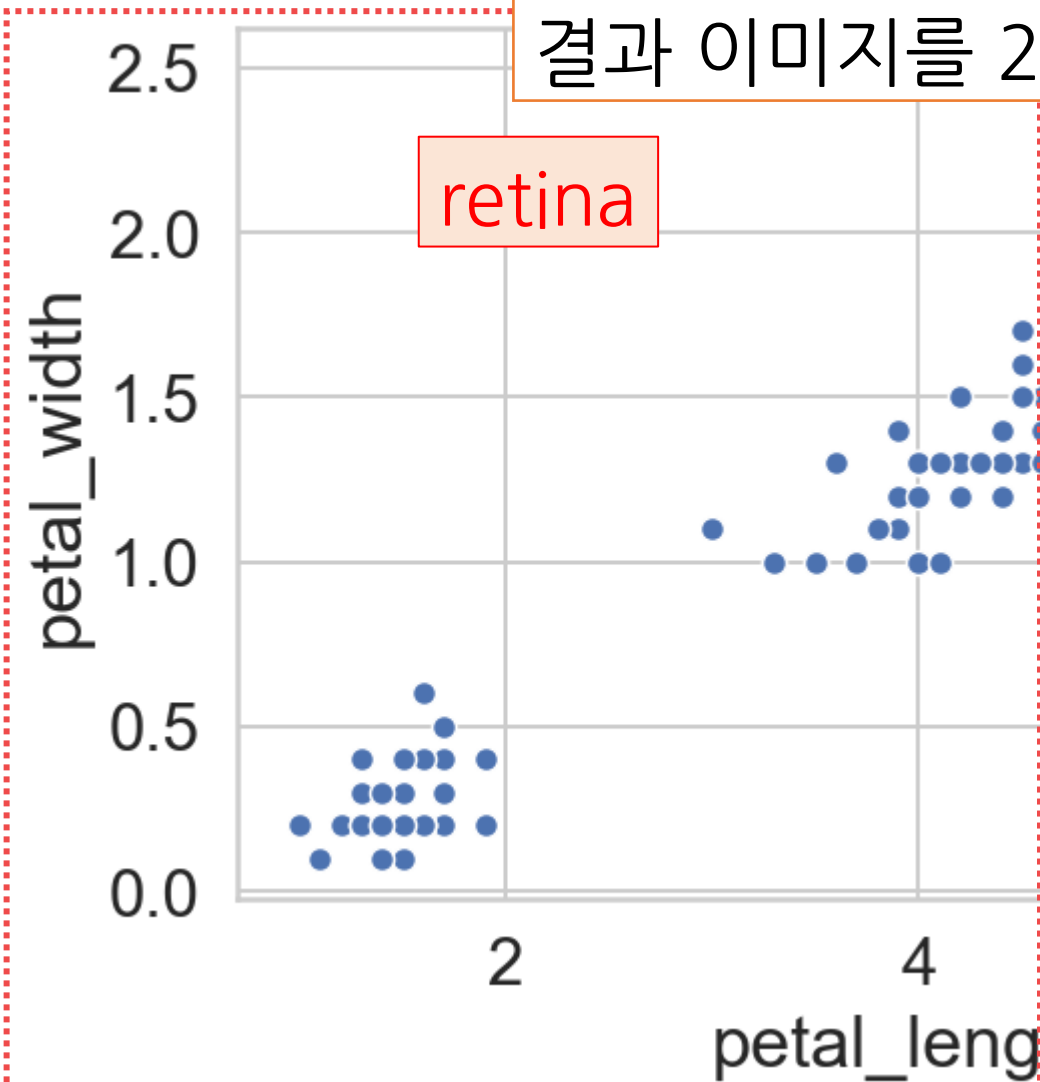
Matplotlib 버전: 3.0.2

```
%config InlineBackend.figure_format = 'retina'
```

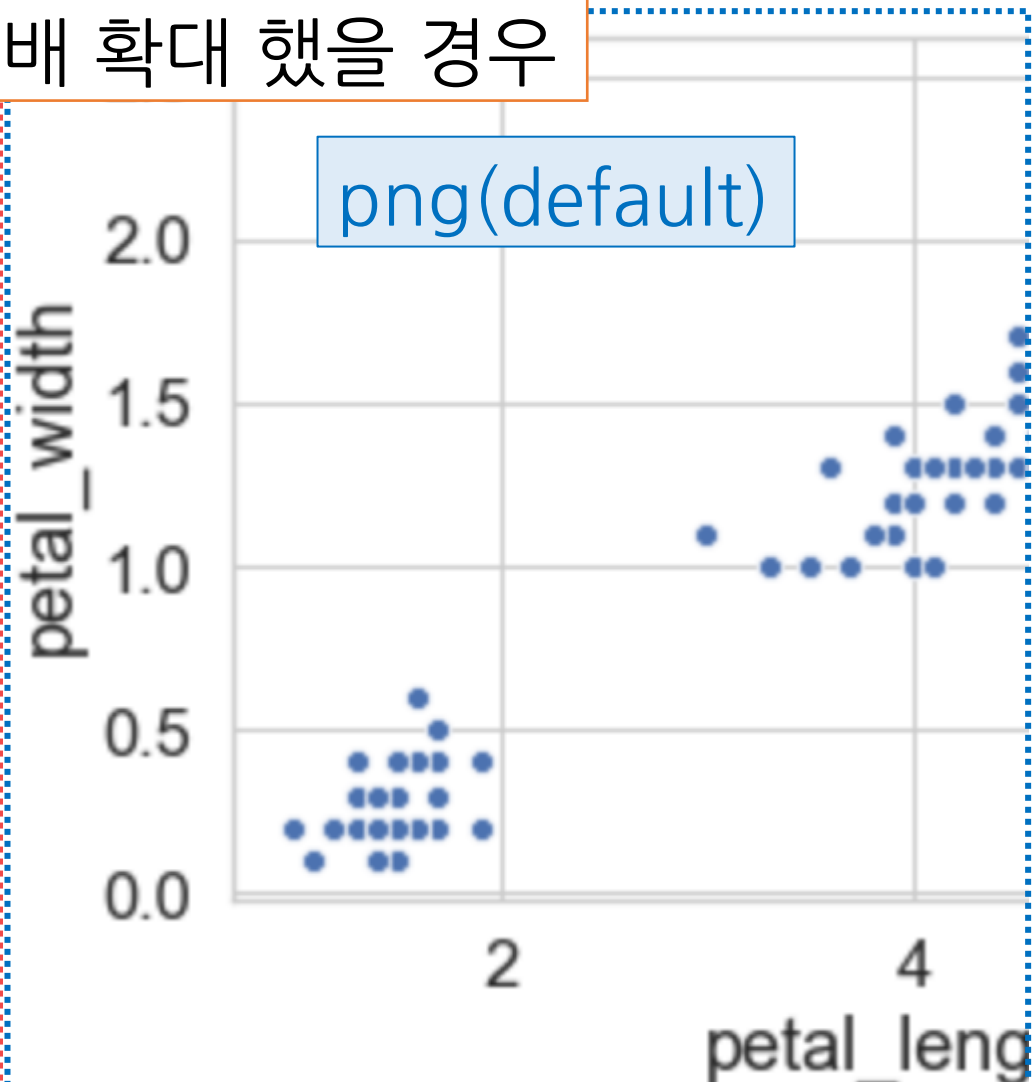
2절. Matplotlib

결과 이미지를 2배 확대했을 경우

retina



png(default)



2.2. 그래프 객체

2절. Matplotlib

- Figure 객체 : Matplotlib에서 **그래프를 그리기 위한 객체**
- 도화지(Figure)를 **plt.subplots()**으로 분할해 각 부분에 **그래프를 그리는 방식**으로 시각화 함

```
matplotlib.pyplot.figure(num=None, figsize=None, dpi=None,
                           facecolor=None, edgecolor=None, frameon=True,
                           FigureClass=<class 'matplotlib.figure.Figure'>,
                           clear=False, **kwargs)
```

- 구문에서...

- num : 정수 또는 문자열, 선택 사항, 기본값 : None. 이 값이 제공되지 않으면 새 그림이 만들어지고 그림 번호가 증가. figure 객체는 숫자 속성에 이 숫자를 저장. num이 제공되고 이 ID를 가진 그림이 이미 존재하면 활성화하고 참조를 반환함. 이 숫자가 없으면 새로 만들어 반환함. num이 문자열이면 윈도우 제목이 설정.
- figsize : (float, float), 선택, 기본값 : None. 너비와 높이를 인치 단위로 지정. 제공되지 않으면 기본값은 plt.rcParams["figure.figsize"]=[6.4, 4.8].

1

```
plt.figure()
```

1) subplot()으로 서브플롯 추가

2절. Matplotlib > 2.3. 그래프 영역 나누기

- subplot() 함수는 현재 figure 객체에 **서브플롯** 추가

```
matplotlib.pyplot.subplot(*args, **kwargs)
```

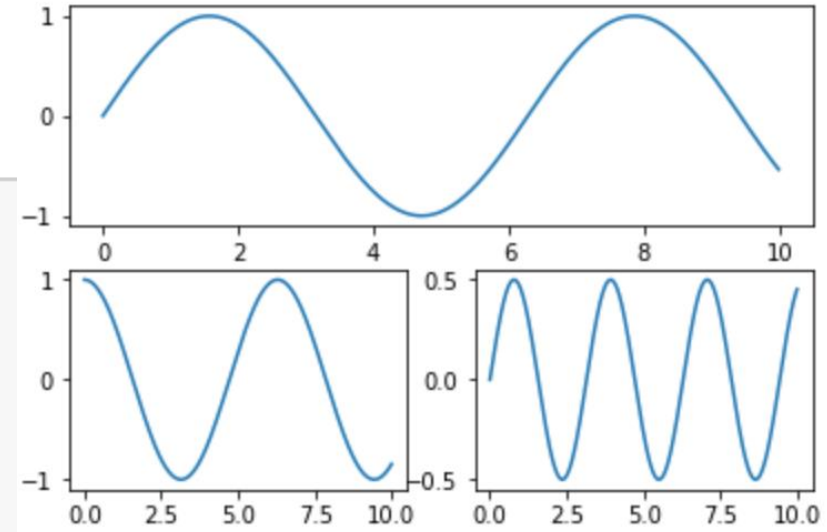
- 구문에서...

- *args*: 서브플롯의 위치를 설명하는 3자리 정수(예: 211) 또는 정수 3개(예: 2,2,1).
- *nrow, ncol, index*: 3개의 정수가 nrow, ncol 및 index 순서로 있는 경우 그려질 하위 그림은 nrow 행과 ncol 열이 있는 표에서 인덱스 위치에 그려짐. index는 왼쪽 상단 모서리에서 1부터 시작하여 오른쪽으로 증가.
- *pos*: pos는 3 자리 정수이며 첫 번째 숫자는 행 수, 두 번째 숫자는 열 수, 세 번째 숫자는 서브 그림의 인덱스. 즉, fig.add_subplot(235)는 fig.add_subplot(2, 3, 5)와 동일. pos 형식이 작동하려면 모든 정수가 10보다 작아야 함. subplot() 함수는 Figure.add_subplot() 함수의 래퍼(Wrapper).
- Returns : 이 함수는 Figure 객체와 axes.Axes 객체(또는 Axes 객체들의 배열)를 반환 함.

1) subplot()으로 서브플롯 추가

2절. Matplotlib > 2.3. 그래프 영역 나누기

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 %matplotlib inline
4
5 x = np.arange(0, 10, 0.01)
6
7 plt.subplot(2, 1, 1); plt.plot(x, np.sin(x))
8 plt.subplot(2, 2, 3); plt.plot(x, np.cos(x))
9 plt.subplot(2, 2, 4); plt.plot(x, np.sin(x)*np.cos(x))
10 plt.show()
```



224와 같음

2) subplots()으로 서브플롯 집합 추가

2절. Matplotlib > 2.3. 그래프 영역 나누기

- subplots() 함수는 현재 figure 객체에 **서브플롯 집합** 추가

```
atplotlib.pyplot.subplots(nrows=1, ncols=1,
                           sharex=False, sharey=False, squeeze=True,
                           subplot_kw=None, gridspec_kw=None, **fig_kw)
```

- 구문에서...

- *nrows, ncols*: int, 옵션, 기본값 : 1. 서브 그래프 그리드의 행/열 수
- *sharex, sharey*: bool 또는 { 'none', 'all', 'row', 'col' }, 기본값 : False.
 x(sharex) 또는 y(sharey) 축 사이의 공유 제어 속성
 . True 또는 'all': x 축 또는 y 축은 모든 하위 플롯간에 공유
 . False 또는 'none': 각 서브플롯 x 또는 y 축은 독립적
 . 'row': 각 subplot 행은 x 또는 y 축을 공유
 . 'col': 각 서브 플롯 열은 x 또는 y 축을 공유
 하위 플롯이 열을 따라 공유 x축을 가질 경우 맨 아래 서브 도표의 x눈금 레이블만 작성됨. 마찬가지로, 하위 그림이 행에 따라 공유 된 y축을 가질 경우 첫 번째 열 subplot의 y눈금 레이블만 만들어짐.

2) subplots()으로 서브플롯 집합 추가

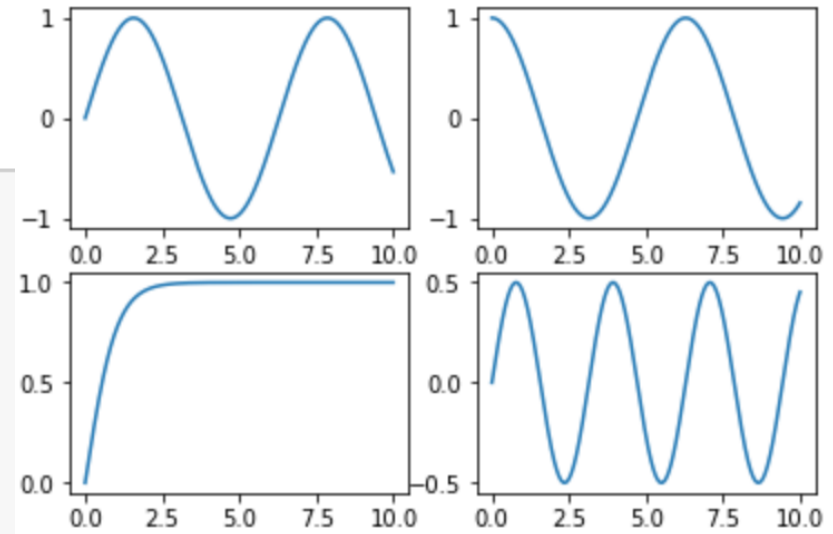
2절. Matplotlib > 2.3. 그래프 영역 나누기

- figure 객체와 axes.Axes(Axes 객체 또는 Axes 객체들 배열)을 반환합니다

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  %matplotlib inline
4
5  x = np.arange(0, 10, 0.01)
6
7  fig, axes = plt.subplots(nrows=2,ncols=2)
8
9  axes[0,0].plot(x, np.sin(x)); axes[0,1].plot(x, np.cos(x))
10 axes[1,0].plot(x, np.tanh(x)); axes[1,1].plot(x, np.sin(x)*np.cos(x))
11 plt.show()

```

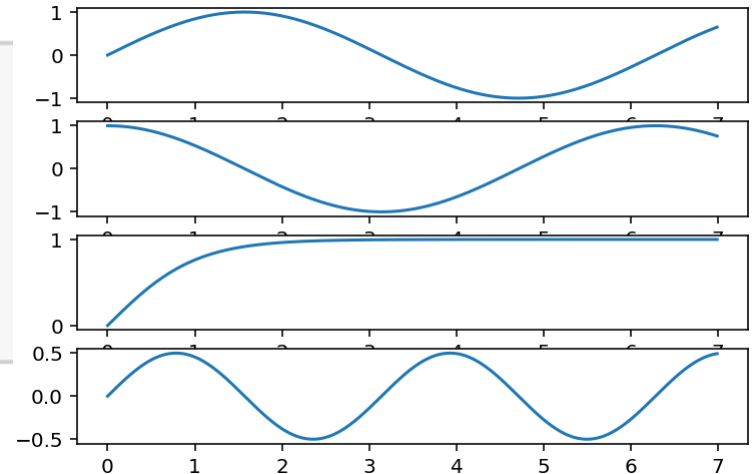


plot함수에서 색상이나 스타일을 조정 : <https://wikidocs.net/92085>

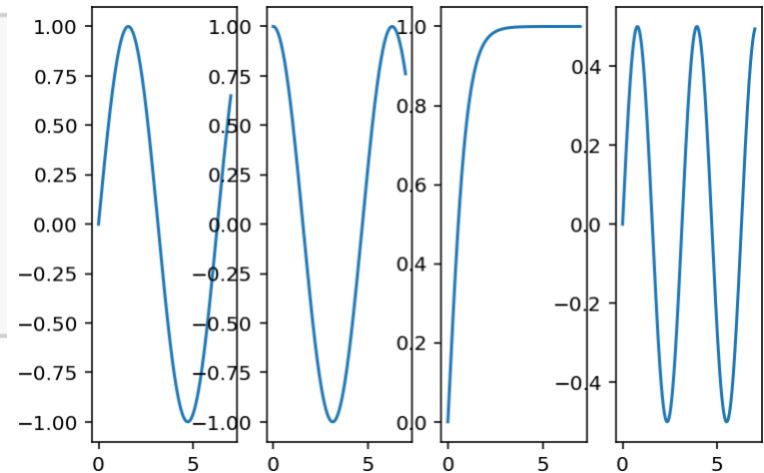
2) subplots()으로 서브플롯 집합 추가

2절. Matplotlib > 2.3. 그래프 영역 나누기

```
1 fig, axes = plt.subplots(nrows=4)
2 for i, ax in enumerate(axes.flat):
3     ax.plot(x, func_list[i](x))
4 plt.show()
```



```
1 fig, axes = plt.subplots(ncols=4)
2 for i, ax in enumerate(axes.flat):
3     ax.plot(x, func_list[i](x))
4 plt.show()
```



1) pyplot 함수들

2절. Matplotlib > 2.4. 그래프그리기

- https://matplotlib.org/api/_as_gen/matplotlib.pyplot.html

matplotlib.pyplot

`matplotlib.pyplot` is a state-based interface to matplotlib. It provides a MATLAB-like way of plotting.

`pyplot` is mainly intended for interactive plots and simple cases of programmatic plot generation:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
plt.plot(x, y)
```

The object-oriented API is recommended for more complex plots.

Functions

<code>acorr(x, *[, data])</code>	Plot the autocorrelation of <i>x</i> .
<code>angle_spectrum(x[, Fs, Fc, window, pad_to, ...])</code>	Plot the angle spectrum.
<code>annotate(s, xy, *args, **kwargs)</code>	Annotate the point <i>xy</i> with text <i>s</i> .
<code>arrow(x, y, dx, dy, **kwargs)</code>	Add an arrow to the axes.
<code>autoscale([enable, axis, tight])</code>	Autoscale the axis view to the data (toggle).
<code>autumn()</code>	Set the colormap to "autumn".
<code>axes([arg])</code>	Add an axes to the current figure and make it the current axes.
<code>axhline([y, xmin, xmax])</code>	Add a horizontal line across the axis.
<code>axhspan(ymin, ymax[, xmin, xmax])</code>	Add a horizontal span (rectangle) across the axis.
<code>axis(*v, **kwargs)</code>	Convenience method to get or set some axis properties.

URL을 참고하세요.

2) plot()

2절. Matplotlib > 2.4. 그래프그리기

- plot() 함수는 주어진 x, y 값을 선(lines)과 점(markers)으로 표시해 줌

```
matplotlib.pyplot.plot([x], y, [fmt], data=None, **kwargs)
```

- 구문에서...
 - *fmt*: 색, 점, 라인의스타일을 문자열로 지정. 예를 들면 'ro-'는 빨간색 동그란 점을 갖는 실선.
 - 점의 모양은 o(원), s(네모), v(역삼각형), ^(삼각형), x(x표시) 등
 - 선의 스타일은 '-'(실선), '--'(대시선), '-.'(대시닷선), ':'(점선), ''(선없음) 등

2) plot()

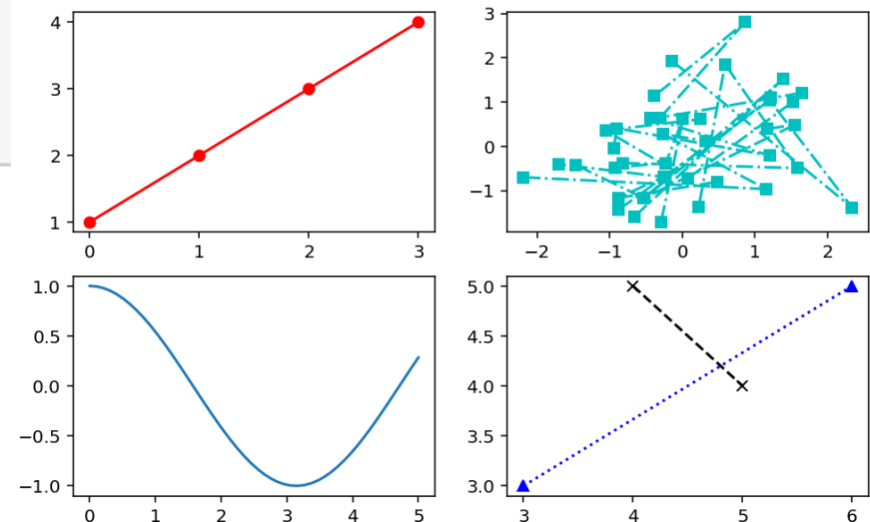
2절. Matplotlib > 2.4. 그래프그리기

```

1 import numpy as np
2 fig, axes = plt.subplots(2,2,figsize=(8,5))
3 fig.suptitle("Figure Sample Plots")
4 axes[0,0].plot([1,2,3,4], 'ro-')
5 axes[0,1].plot(np.random.randn(4,10),
6                np.random.randn(4,10), 'cs-.')
7 axes[1,0].plot(np.linspace(0,5),
8                np.cos(np.linspace(0,5)))
9 axes[1,1].plot([3,6], [3,5], 'b^:')
10 axes[1,1].plot([4,5], [5,4], 'kx--')
11 plt.show()

```

Figure Sample Plots



3) pandas.DataFrame.plot()

2절. Matplotlib > 2.4. 그래프그리기

- 판다스의 데이터프레임 객체를 이용해서 그래프를 그릴 수 있음
- 데이터프레임의 plot() 함수는 데이터프레임의 데이터를 쉽게 시각화

```
DataFrame.plot(x=None, y=None, kind='line', ax=None, subplots=False,
               sharex=None, sharey=False, layout=None, figsize=None,
               use_index=True, title=None, grid=None, legend=True, style=None,
               logx=False, logy=False, loglog=False, xticks=None, yticks=None,
               xlim=None, ylim=None, rot=None, fontsize=None, colormap=None,
               table=False, yerr=None, xerr=None, secondary_y=False,
               sort_columns=False, **kwds)
```

- 구문에서...
 - kind : 그래프의 종류('line', 'bar', 'barh', 'hist', 'box', 'kde', 'density', 'area', 'pie', 'scatter', 'hexbin')

3) pandas.DataFrame.plot()

2절. Matplotlib > 2.4. 그래프그리기

```

1  import numpy as np
2  import pandas as pd
3
4  import statsmodels.api as sm
5  iris = sm.datasets.get_rdataset("iris", package="datasets")
6  iris_df = iris.data
7  iris_df.head()

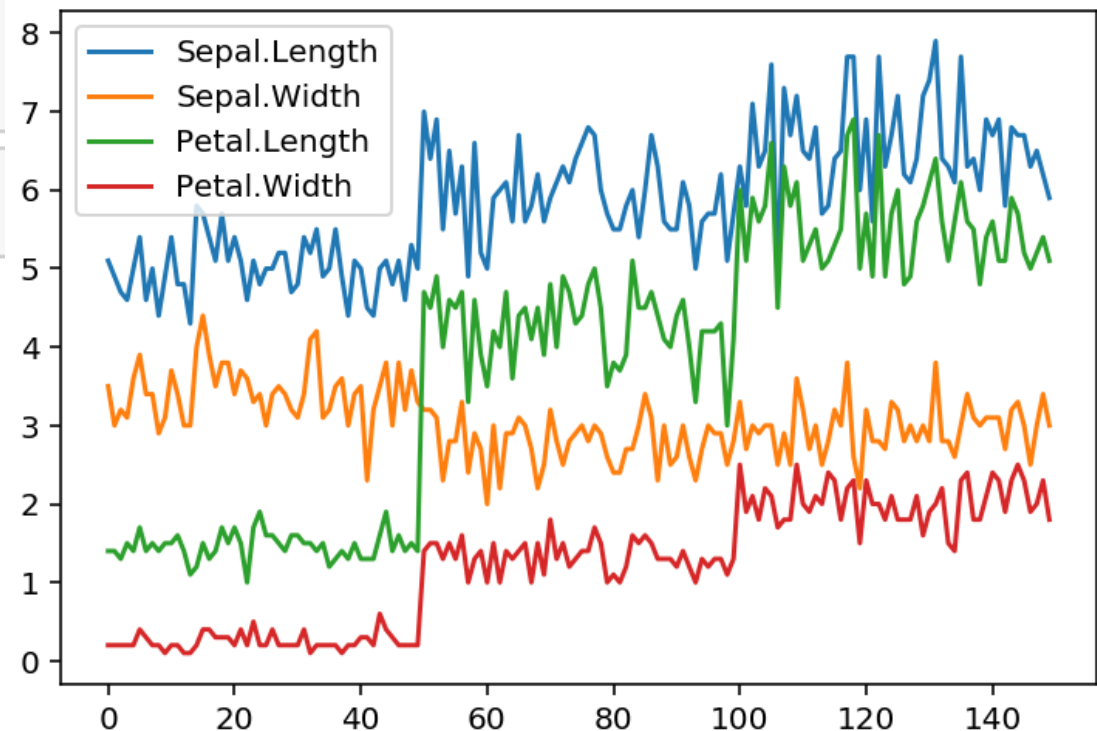
```

```

1  iris_df.plot()

```

<https://stackoverflow.com/questions/30490740/move-legend-outside-figure-in-seaborn-tsplo> :
범례사용



3) pandas.DataFrame.plot()

2절. Matplotlib > 2.4. 그래프그리기

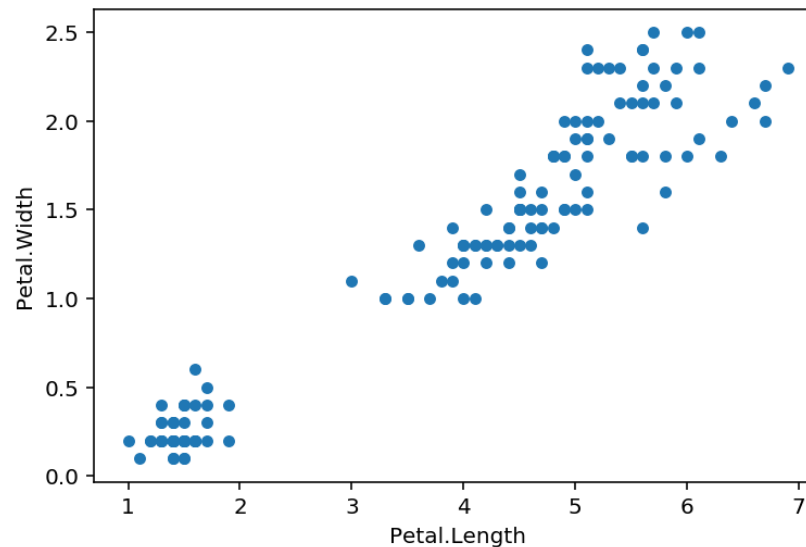
```
1 iris_df.corr()
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Sepal.Length	1.000000	-0.117570	0.871754	0.817941
Sepal.Width	-0.117570	1.000000	-0.428440	-0.366126
Petal.Length	0.871754	-0.428440	1.000000	0.962865
Petal.Width	0.817941	-0.366126	0.962865	1.000000

상관계수를 출력하고 상관관계가 높은 두 변수를 이용해서 산점도 그래프를 그려봄

```
1 iris_df.plot(x="Petal.Length",y='Petal.Width',kind='scatter')
```

<matplotlib.axes._subplots.AxesSubplot at 0x2ca92ce1710>



https://ko.wikipedia.org/wiki/상자_수염_그림

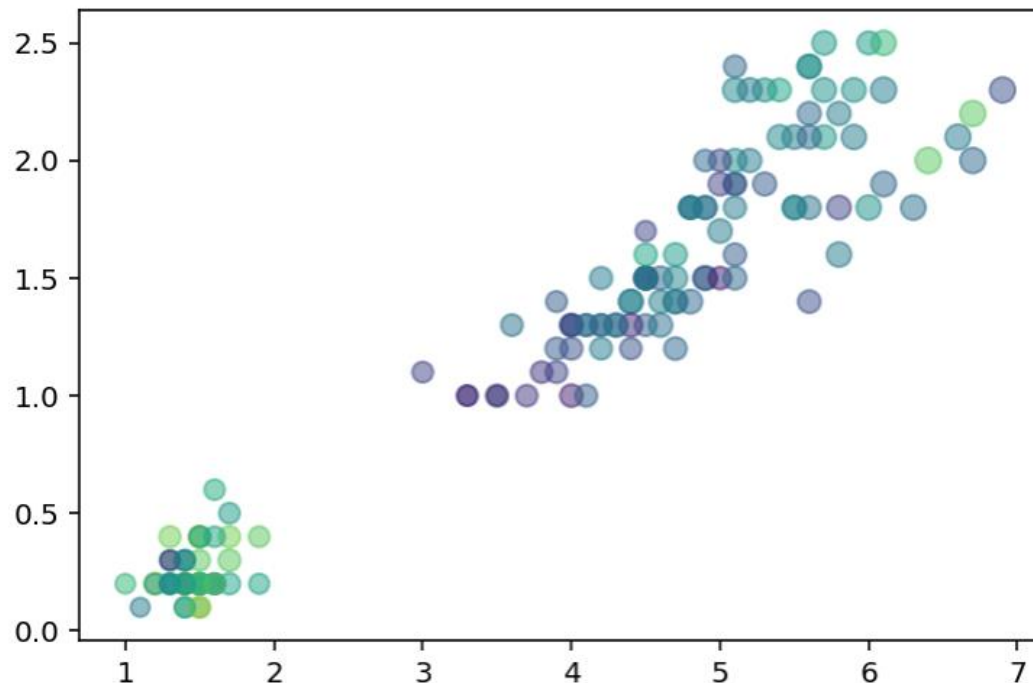
<https://matplotlib.org/stable/tutorials/colors/colormaps.html> (colormap)

4) scatter()

2절. Matplotlib > 2.4. 그래프그리기

- scatter() 함수는 산점도 그래프를 그려줌

```
1 plt.scatter(x=iris.petal_length, y=iris.petal_width,  
2             s=iris.sepal_length*10, c=iris.sepal_width,  
3             alpha=0.5)  
4 plt.show()
```



3) scatter()

2절. Matplotlib > 2.4. 그래프그리기

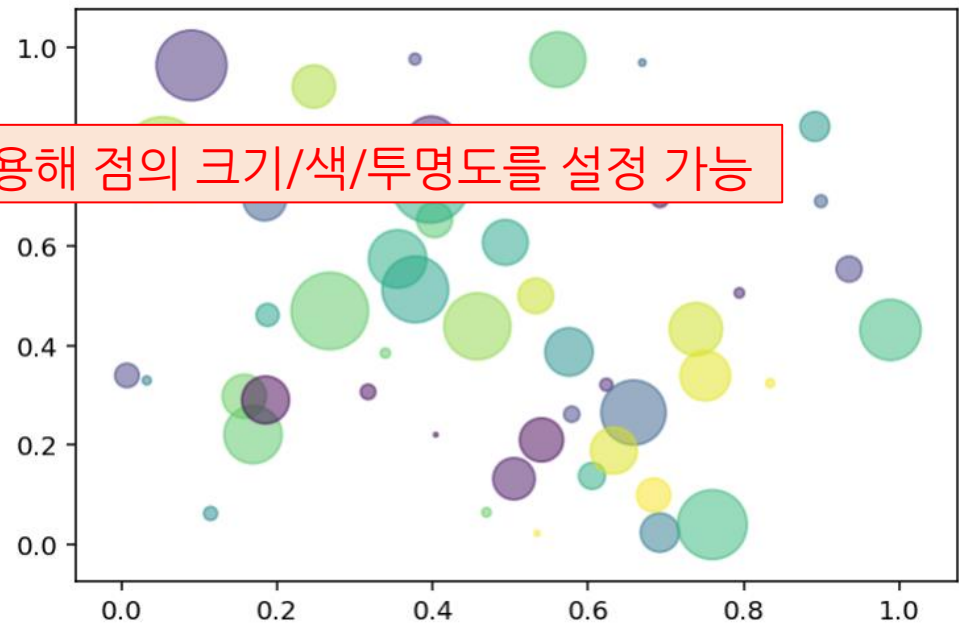
https://matplotlib.org/api/_as_gen/matplotlib.pyplot.scatter.html

- scatter() 함수는 산점도 그래프를 그려줌

```
matplotlib.pyplot.scatter(x, y, s=None, c=None,  
                           marker=None, cmap=None, norm=None, vmin=None,  
                           vmax=None, alpha=None, linewidths=None,  
                           verts=None, edgecolors=None, *, data=None,  
                           **kwargs)
```

```
1 import numpy as np  
2 import matplotlib.pyplot as plt  
3  
4 np.random.seed(7902)  
5  
6 N = 50  
7 x = np.random.rand(N)  
8 y = np.random.rand(N)  
9 colors = np.random.rand(N)  
10 area = (30 * np.random.rand(N))**2  
11  
12 plt.scatter(x, y, s=area, c=colors, alpha=0.5)  
13 plt.show()
```

데이터를 이용해 점의 크기/색/투명도를 설정 가능

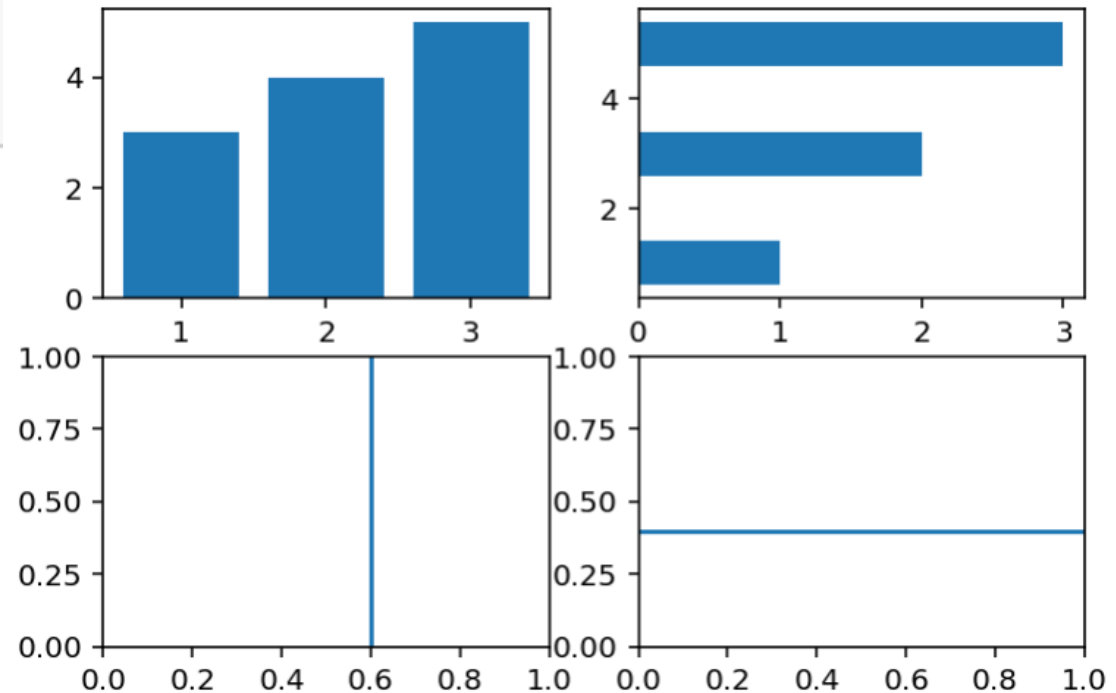


4) bar(), barh(), axvline(), axhline()

2절. Matplotlib > 2.4. 그래프그리기

```
1 fig, axes = plt.subplots(2,2)
2 axes[0,0].bar([1,2,3], [3,4,5])
3 axes[0,1].barh([1,3,5], [1,2,3])
4 axes[1,0].axvline(0.6)
5 axes[1,1].axhline(0.4)
6 plt.show()
```

- bar() 함수는 수직 막대그래프
- barh() 함수는 수평 막대그래프
- axhline()은 수평선, axvline()은 수직선



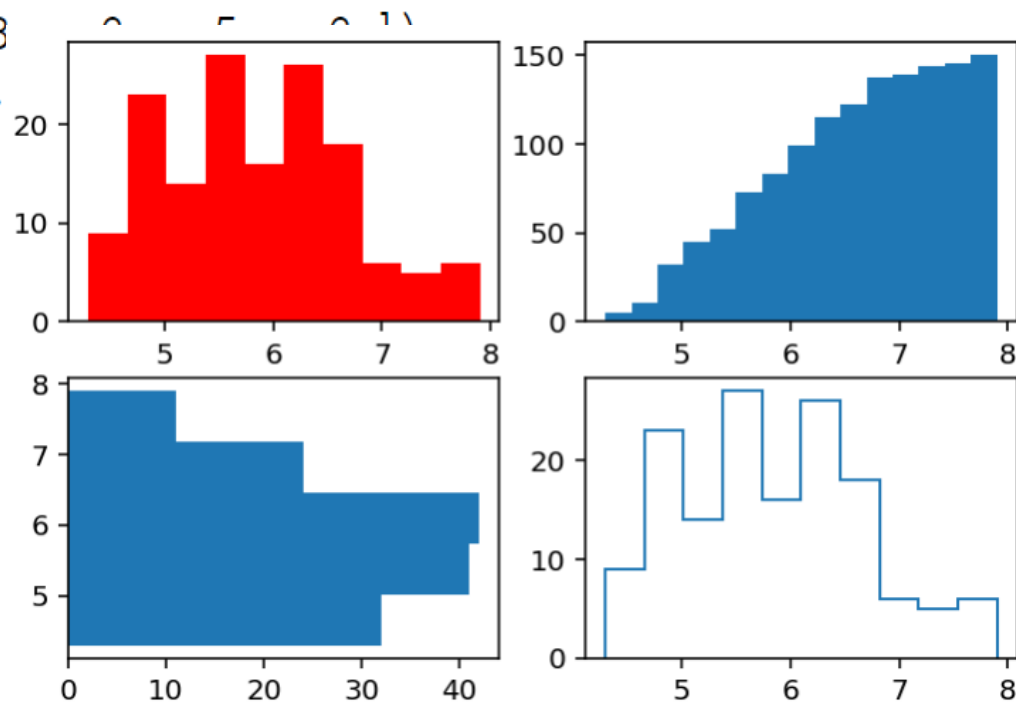
5) hist()

2절. Matplotlib > 2.4. 그래프그리기

```
1 fig, axes = plt.subplots(2,2)
2 axes[0,0].hist(iris_df["sepal_length"], bins=10, color='r')
3 axes[0,1].hist(iris_df["sepal_length"], bins=15, cumulative=True)
4 axes[1,0].hist(iris_df["sepal_length"], bins=5, orientation='horizontal')
5 axes[1,1].hist(iris_df["sepal_length"], bins=10, histtype='step')
```

```
(array([ 9., 23., 14., 27., 16., 26., 18
array([4.3 , 4.66, 5.02, 5.38, 5.74, 6.
<a list of 1 Patch objects>)
```

- 데이터의 분포를 확인할 수 있는 가장 좋은 방법 중 하나는 히스토그램을 그리는 것
- hist() 함수는 히스토그램을 그려주며 동시에 히스토그램 값을 함께 반환



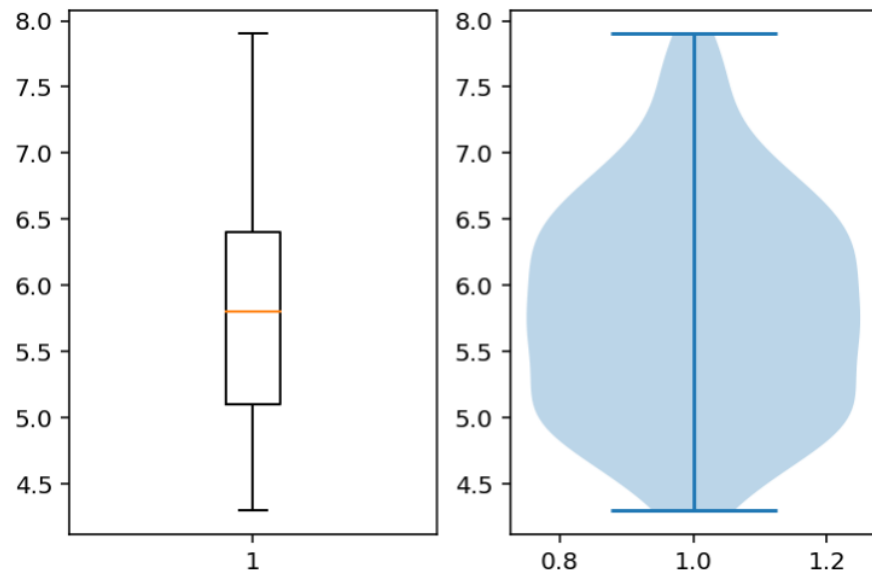
6) boxplot(), violineplot()

2절. Matplotlib > 2.4. 그래프그리기

```
1 fig, axes = plt.subplots(1,2)
2 axes[0].boxplot(iris_df["sepal_length"])
3 axes[1].violinplot(iris_df["sepal_length"])
```

```
{'bodies': [<matplotlib.collections.PolyCollection at 0x256cbf2a518>],
'cmaxes': <matplotlib.collections.LineCollection at 0x256cbf14978>,
'cmins': <matplotlib.collections.LineCollection at 0x256cbf2a9e8>,
'cbars': <matplotlib.collections.LineCollection at 0x256cbf2ac50>}
```

- 사분위수 그래프와 바이올린 그래프는 구간 별 데이터의 분포를 확인



7) fill(), fill_between(), fill_betweenx()

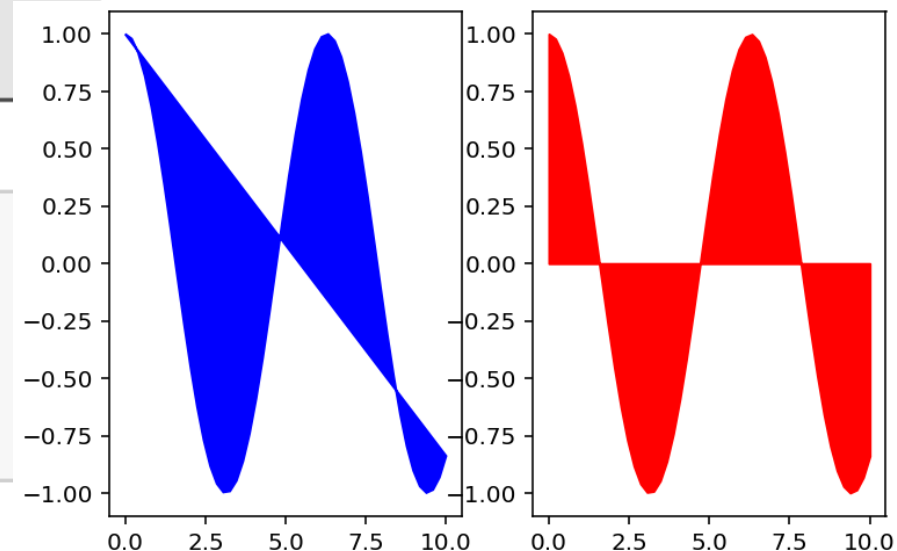
2절. Matplotlib > 2.4. 그래프그리기

```
matplotlib.pyplot.fill(*args, data=None, **kwargs)
```

```
matplotlib.pyplot.fill_between(x, y1, y2=0, where=None,
                               interpolate=False, step=None, *, data=None,
                               **kwargs)
```

```
matplotlib.pyplot.fill_betweenx(y, x1, x2=0, where=None,
                                step=None, interpolate=False, *, data=None,
                                **kwargs)
```

```
1 fig, axes = plt.subplots(1,2)
2 axes[0].fill(x, y, c='blue')
3 axes[1].fill_between(x, y, color='red')
4 plt.show()
```



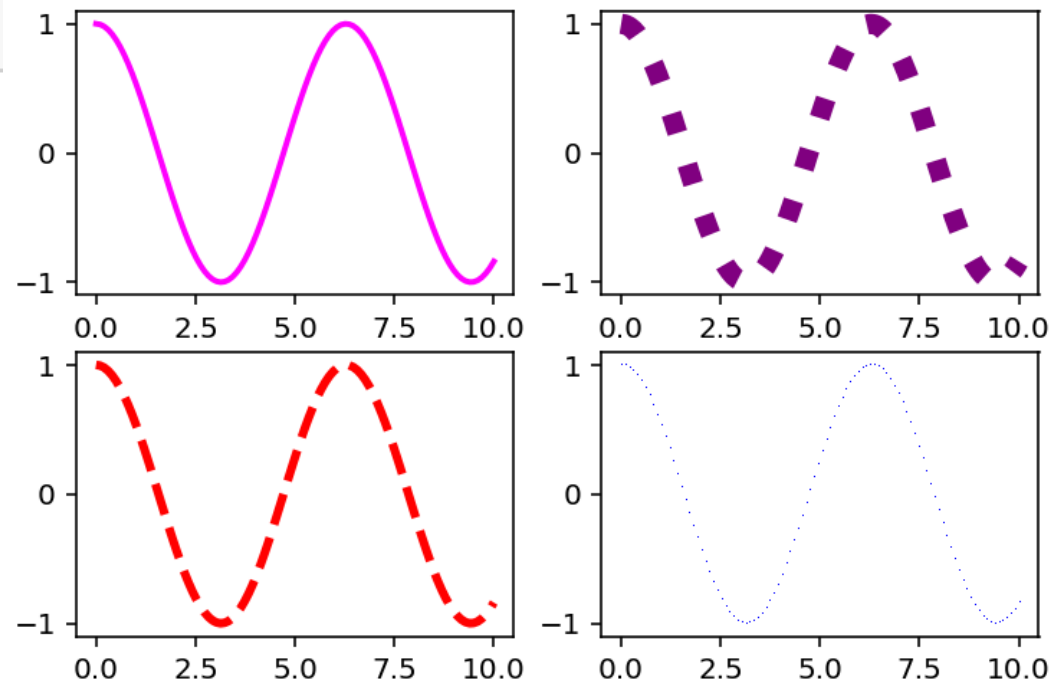
1) linestyle, linewidth

2절. Matplotlib > 2.5. 그래프 커스터마이징

```

1 fig, axes = plt.subplots(2,2)
2 axes[0,0].plot(x, y, linewidth=2, color="#FF00FF")
3 axes[0,1].plot(x, y, linestyle="dotted", linewidth=7, color="purple")
4 axes[1,0].plot(x, y, ls="--", c="r", lw=3)
5 axes[1,1].plot(x, y, "b.") # 1 픽셀 파란색
6 plt.show()

```



2) text(), annotate()

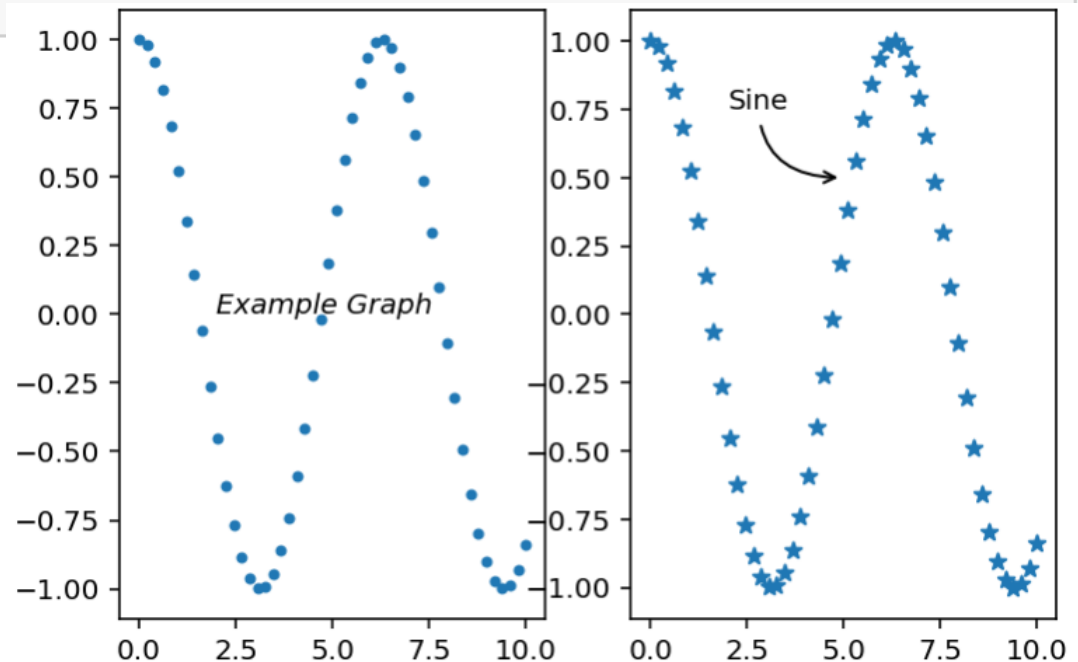
2절. Matplotlib > 2.5. 그래프

참고 : https://matplotlib.org/api/axes_api.html#text-and-annotations

```

1 fig, axes = plt.subplots(1,2)
2 axes[0].scatter(x, y, marker=".")
3 axes[0].text(2, 0, 'Example Graph', style='italic')
4 axes[1].scatter(x, y, marker="*")
5 axes[1].annotate("Sine", xy=(5, 0.5), xytext=(2, 0.75),
6                 arrowprops=dict(arrowstyle="->", connectionstyle="angle3"))
7 plt.show()

```

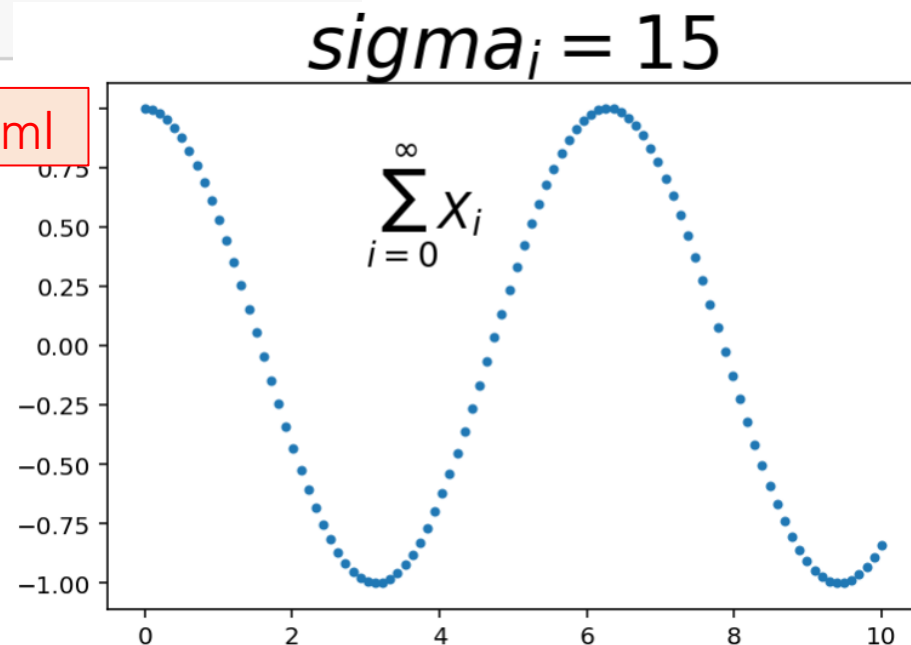


3) 수학 기호

2절. Matplotlib > 2.5. 그래프 커스터마이징

```
1 x = np.linspace(0, 10, 100)
2 y = np.cos(x)
3
4 fig, ax = plt.subplots()
5 ax.scatter(x,y,marker=".")
6 ax.set_title(r"$\sigma_i=15$", fontsize=30)
7 ax.text(3, 0.5, r"$\sum_{i=0}^{\infty} X_i$", fontsize=20)
8 plt.show()
```

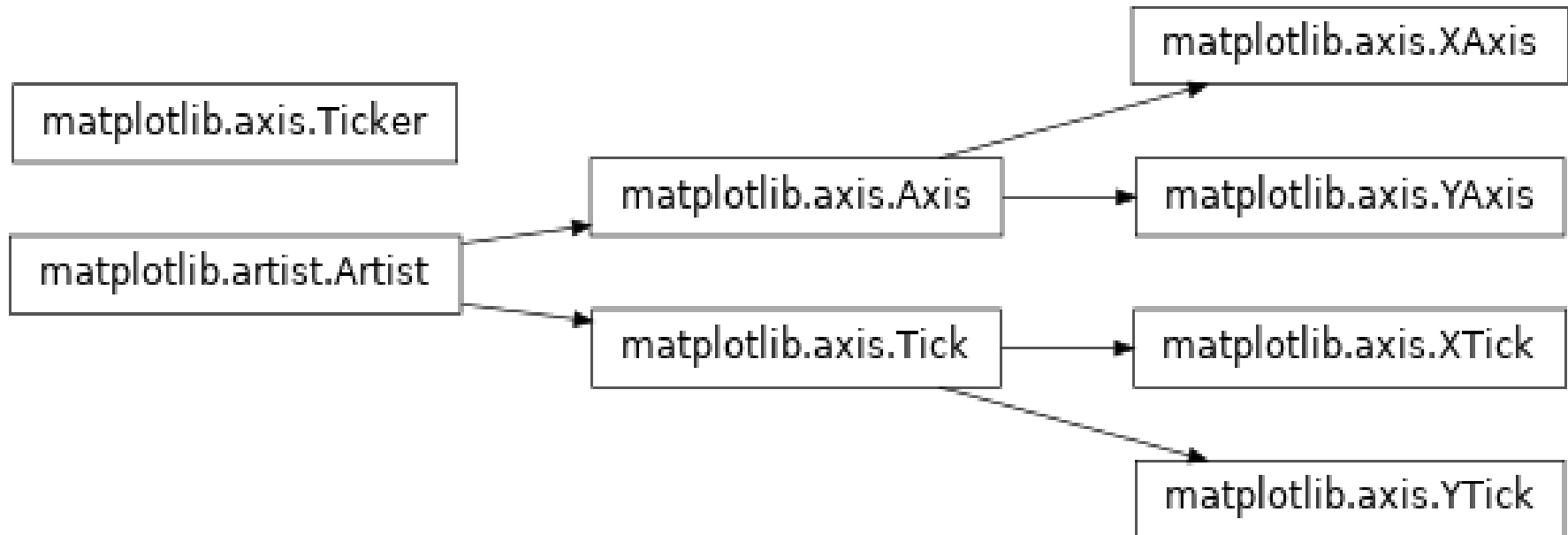
참고 : <https://matplotlib.org/users/mathtext.html>



4) 축과 눈금

2절. Matplotlib > 2.5. 그래프 커스터마이징

- 축(Axis)과 눈금(Tick)은 Artist 클래스를 상속
 - X축과 Y축은 Axis 클래스를 상속
 - X축 눈금과 Y축 눈금은 Tick 클래스를 상속
- 축과 눈금의 지정
 - Axis 클래스의 `set_x???`() 메서드를 이용 X축의 스타일을 지정



4) 축과 눈금

2절. Matplotlib > 2.5. 그래프 커스터마이징

- `set()`은 그래프의 제목과 축의 제목, 눈금, 눈금 레이블 등을 설정
- `set_xlim()`, `set_ylim()`은 각각 X축과 Y축의 범위를 설정
- `set_xlabel()`, `set_ylabel()`은 X축과 Y축의 레이블을 설정
- `set_xticks()`, `set_yticks()`는 눈금 위치를 리스트 형식으로 지정
- `set_xticklabels()`, `set_yticklabels()`는 눈금의 레이블을 리스트 형식으로 지정
- `spines` 속성은 그래프의 박스 경계를 설정
- `grid(True)` 이면 눈금선을 보여줌
- https://matplotlib.org/3.1.0/api/axis_api.html

4) 축과 눈금

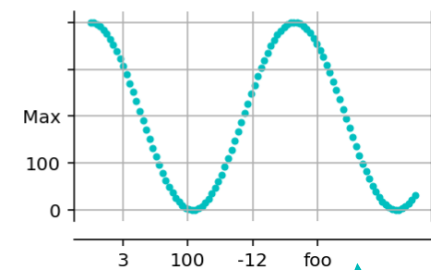
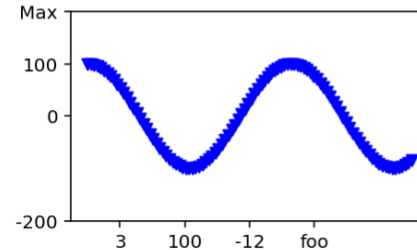
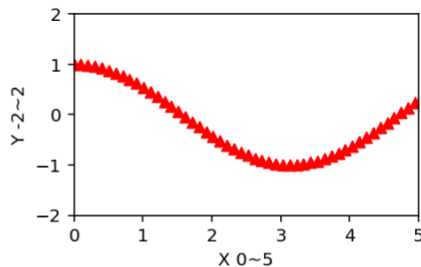
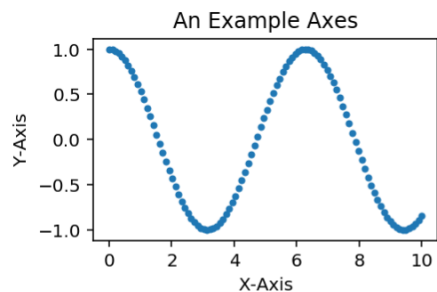
2절. Matplotlib > 2.5. 그래프 커스터마이징

https://matplotlib.org/3.1.0/api/axis_api.html

```

1  import numpy as np
2  x = np.linspace(0, 10, 100)
3  y = np.cos(x)
4
5  fig, axes = plt.subplots(2,2, figsize=(8,5))
6  plt.subplots_adjust(hspace=0.4, wspace=0.3)
7  axes[0,0].scatter(x, y, marker=".")
8  axes[0,0].set(title="An Example Axes", ylabel="Y-Axis", xlabel="X-Axis")
9
10 axes[0,1].scatter(x, y, marker="^", c="r")
11 axes[0,1].set_xlim(0,5)
12 axes[0,1].set_ylim(-2,2)
13 axes[0,1].set_xlabel("X 0~5")
14 axes[0,1].set_ylabel("Y -2~2")
15

```



```

16 axes[1,0].scatter(x, y, marker="v", c="b")
17 axes[1,0].set_xticks(range(1,8,2))
18 axes[1,0].set_xticklabels([3,100,-12,"foo"])
19 axes[1,0].set_yticks([-2,0,1,2])
20 axes[1,0].set_yticklabels([-200,0,100,"Max"])
21
22 axes[1,1].scatter(x, y, marker=".", c="c")
23 axes[1,1].set(xticks=range(1,8,2),
24              xticklabels=[3,100,-12,"foo"],
25              #yticks=[-2,0,1,2],
26              yticklabels=[-200,0,100,"Max"])
27 axes[1,1].spines['top'].set_visible(False)
28 axes[1,1].spines['bottom'].set_position(("outward", 10))
29 axes[1,1].grid(True)
30 plt.show()

```

5) 축 공유

2절. Matplotlib > 2.5. 그래프 커스터마이징

- 하나의 그래프 영역에 두 개 이상의 그래프를 그리면서 다른 축을 지정하고 싶을 경우 `twinx()`, `twiny()` 함수를 이용해서 새로운 축 객체를 생성하여 그래프를 그리고 축을 다르게 지정
- `twinx()` 함수는 x축을 공유하고 y축을 오른쪽에 표시해 주는 새로운 축 객체를 반환
- `twiny()` 함수는 y축을 공유하고 x축을 위쪽에 표시해 주는 새로운 축 객체를 반환

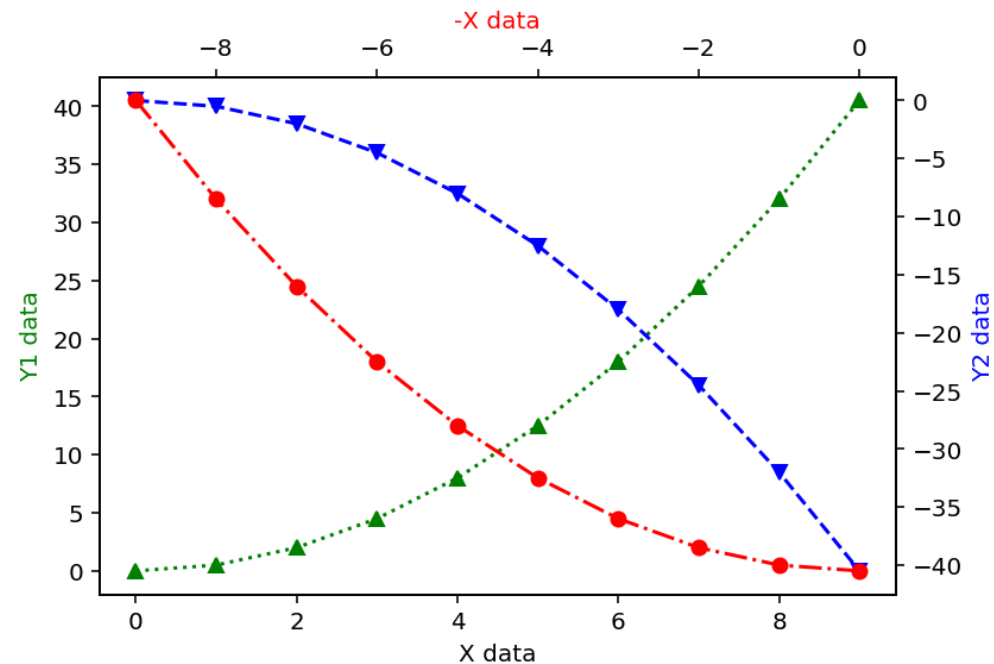
5) 축 공유

2절. Matplotlib > 2.5. 그래프 커스터마이징

```

1  x = np.arange(0, 10)
2  y1 = 0.5 * x**2
3  y2 = -1 * y1
4
5  fig, ax1 = plt.subplots()
6  ax1.plot(x, y1, 'g^:')
7  ax1.set_xlabel('X data')
8  ax1.set_ylabel('Y1 data', color='g')
9
10 ax2 = ax1.twinx()
11 ax2.plot(x, y2, 'bv--')
12 ax2.set_ylabel('Y2 data', color='b')
13
14 ax3 = ax1.twinx()
15 ax3.plot(-x, y1, 'ro-.')
16 ax3.set_xlabel('-X data', color='r')
17
18 plt.show()

```



6) 그래프 제목과 축 제목

2절. Matplotlib > 2.5. 그래프 커스터마이징

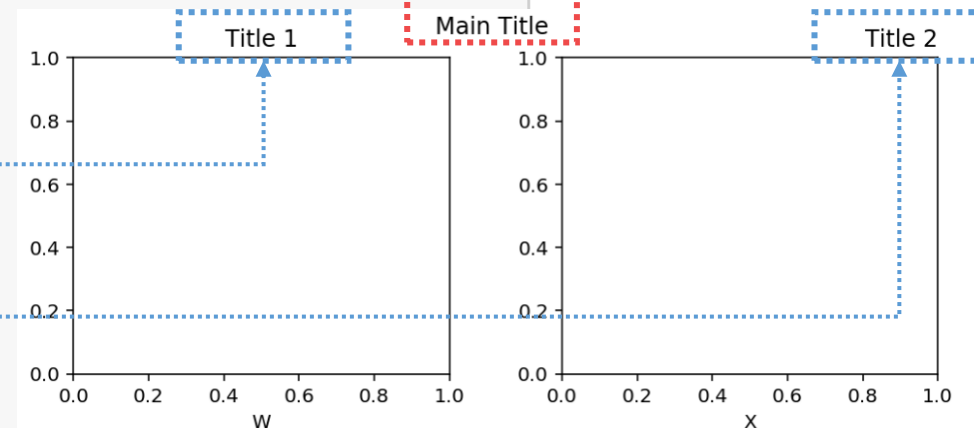
- `plot.suptitle()` 함수는 그래프의 제목을 설정

```
matplotlib.pyplot.suptitle(t, **kwargs)
```

- `axes.Axes.set_title()` 함수는 그래프 영역 안에서 축의 제목을 설정

```
matplotlib.axes.Axes.set_title(label, fontdict=None, loc='center, pad=None, **kwargs)
```

```
1 fig, axes = plt.subplots(1,2, figsize=(8,3))
2 plt.subplots_adjust(hspace=0.4, wspace=0.3)
3 plt.suptitle("Main Title")
4
5 axes[0].set_title("Title 1")
6 axes[0].set_xlabel("W")
7
8 axes[1].set_title("Title 2", loc='right')
9 axes[1].set_xlabel("X")
10 plt.show()
```



7) 범례 표시

2절. Matplotlib > 2.5. 그래프 커스터마이징

- 범례를 표시하는 가장 좋은 방법은 그래프를 그리는 함수에 의해 범례가 자동으로 그려지게 하는 방법

```
matplotlib.pyplot.legend()  
matplotlib.pyplot.legend(labels)
```

- `legend()` 함수는 `plot()` 함수가 `label` 속성을 가질 경우 함수의 호출만으로 범례를 표시
- `plot()` 함수에 `label` 속성을 지정하지 않을 경우 `legend()` 함수의 `labels` 파라미터를 이용해서 범례의 레이블(이름)을 지정

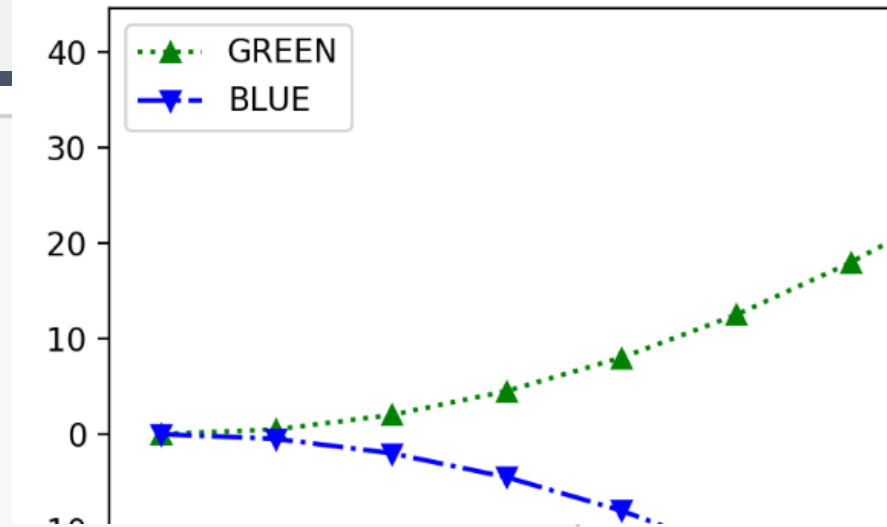
7) 범례 표시

2절. Matplotlib > 2.5. 그래프 커스터마이징

```

1  import numpy as np
2  x = np.arange(0,10)
3  y = 0.5 * x**2
4
5  plt.style.use('default')
6  fig, ax1 = plt.subplots()
7  ax1.plot(x, y, 'g^:', label='GREEN')
8  ax1.plot(x, -y, 'bv-.', label='BLUE')
9  ax1.legend()
10 plt.show()

```



그래프 함수가 label을 가질 경우

```

7  ax1.plot(x, y, 'g^:')
8  ax1.plot(x, -y, 'bv-.')
9  ax1.legend(labels=['GREEN', 'BLUE'])
10 plt.show()

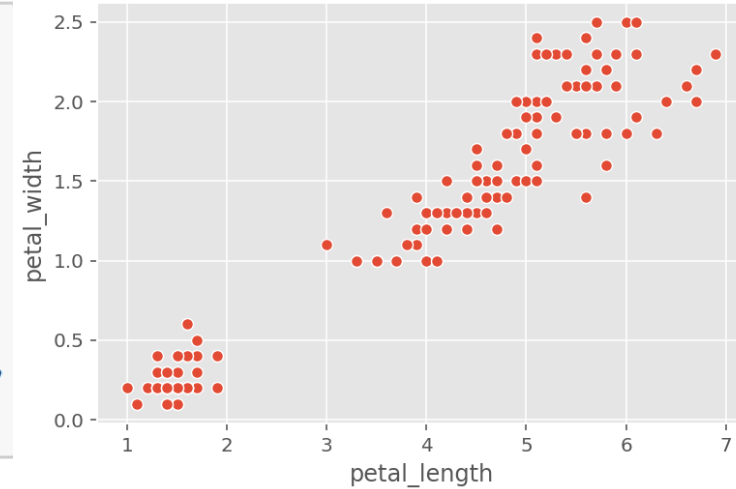
```

그래프 함수가 label을
가지 않을 경우

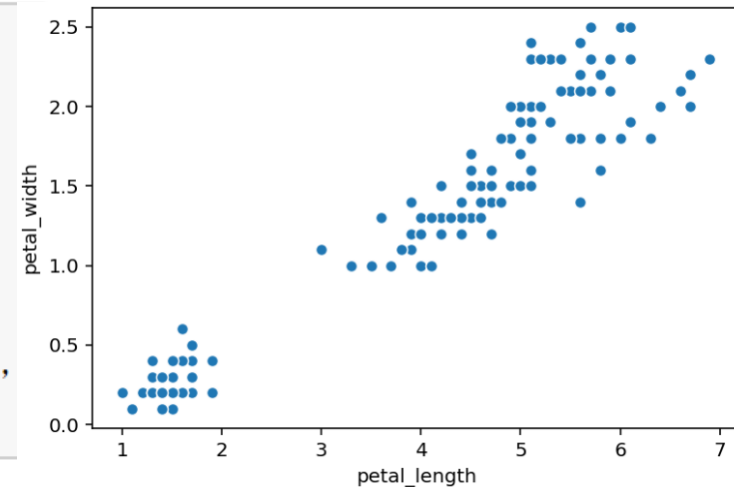
9) 플롯 스타일 지정

2절. Matplotlib > 2.5. 그래프 커스터마이징

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 %matplotlib inline
4 iris = sns.load_dataset("iris")
5 fig = plt.figure()
6 plt.style.use('ggplot')
7 ax = sns.scatterplot(x="petal_length", y="petal_width",
8                     data=iris)
```



```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 %matplotlib inline
4 iris = sns.load_dataset("iris")
5 fig = plt.figure()
6 plt.style.use('default')
7 ax = sns.scatterplot(x="petal_length", y="petal_width",
8                     data=iris)
```

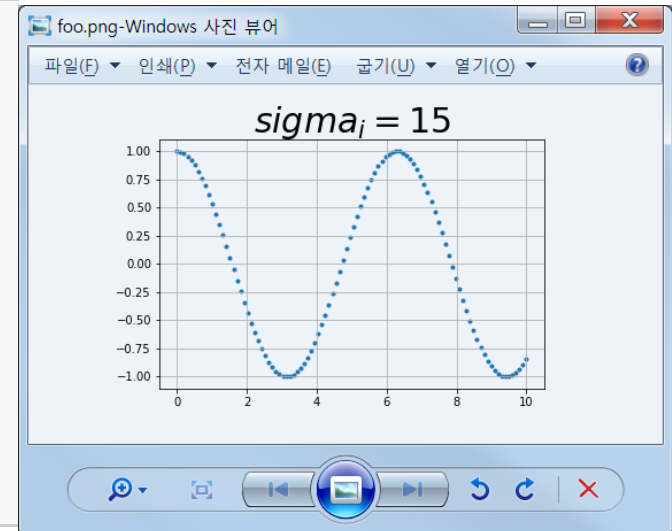


2.7. 그래프 저장

2절. Matplotlib

```
savefig(fname, dpi=None, facecolor='w', edgecolor='w',
        orientation='portrait', papertype=None, format=None,
        transparent=False, bbox_inches=None, pad_inches=0.1,
        frameon=None, metadata=None)
```

```
1 x = np.linspace(0, 10, 100)
2 y = np.cos(x)
3
4 fig, ax = plt.subplots()
5 ax.scatter(x, y, marker='.')
6 ax.set_title(r"$\sigma_i=15$", fontsize=30)
7 ax.grid(True)
8 plt.savefig("foo.png")
```



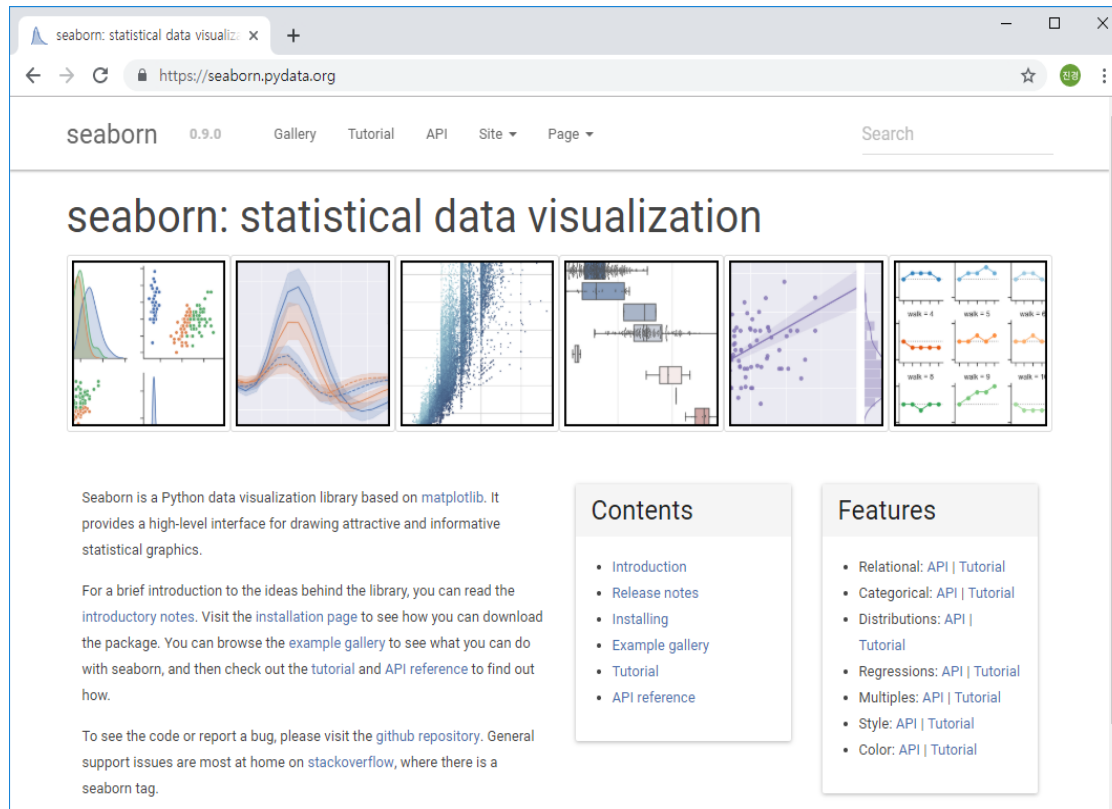
3절. Seaborn

- matplotlib을 기반으로 만든 고수준 그래픽 라이브러리
 - 공식사이트 : <https://seaborn.pydata.org>
 - seaborn API : <https://seaborn.pydata.org/api.html>
- Seaborn으로 그래프를 그리기 위해서 다음 단계를 따릅니다.
 - 1) 데이터 준비
 - 2) 미적속성 설정
 - 3) 함수를 이용하여 그래프 그리기
 - 4) 그래프 출력, 저장

3절. Seaborn

3절. Seaborn

- matplotlib를 기반으로 만들어진 고수준 그래프 라이브러리
 - 공식 사이트 : <https://seaborn.pydata.org/>
 - 그래프 API : <https://seaborn.pydata.org/api.html>



3.1. 데이터 준비하기

3절. Seaborn

- Seaborn의 내장 데이터셋
- `iris = sns.load_dataset("iris")`

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

- `titanic = sns.load_dataset("titanic")`

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

1) 그래프 영역 설정하기

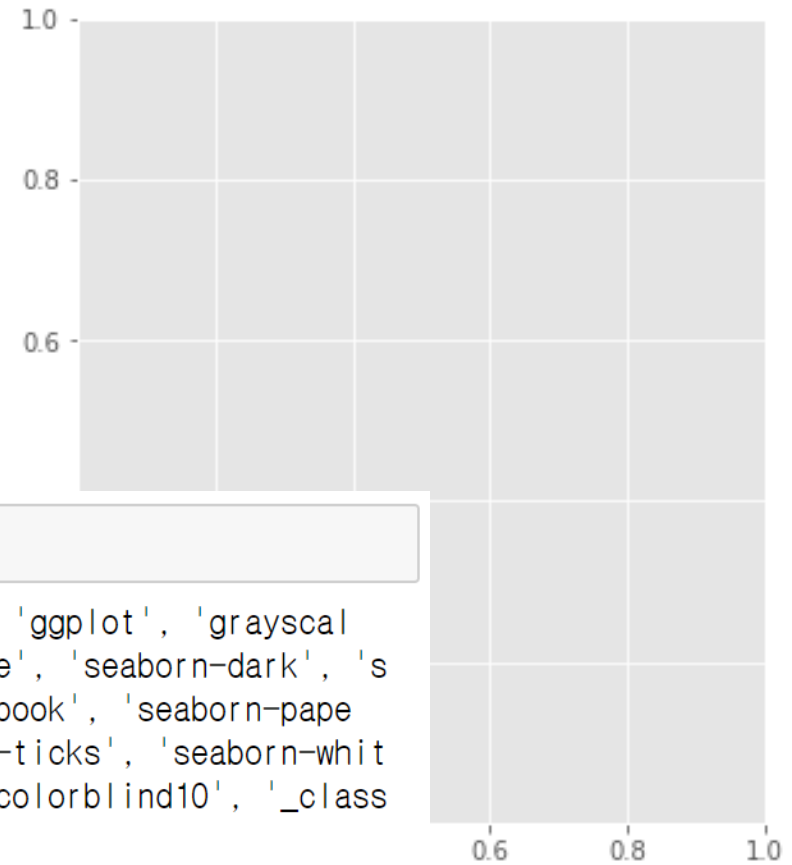
3절. Seaborn > 3.2. 미적 속성 설정하기

- Seaborn으로 그래프를 그리기 위한 그래프 영역은 matplotlib.pyplot 모듈의 `subplots()`를 이용

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 %matplotlib inline
4 plt.style.use('ggplot')
5 fig, ax = plt.subplots(figsize=(5,6))
6 plt.show()
```

```
1 print(plt.style.available)
```

```
['bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn-bright', 'seaborn-colorblind', 'seaborn-dark-palette', 'seaborn-dark', 'seaborn-darkgrid', 'seaborn-deep', 'seaborn-muted', 'seaborn-notebook', 'seaborn-paper', 'seaborn-pastel', 'seaborn-poster', 'seaborn-talk', 'seaborn-ticks', 'seaborn-white', 'seaborn-whitegrid', 'seaborn', 'Solarize_Light2', 'tableau-colorblind10', '_classic_test']
```



2) Seaborn 스타일 지정

3절. Seaborn > 3.2. 미적 속성 설정하기

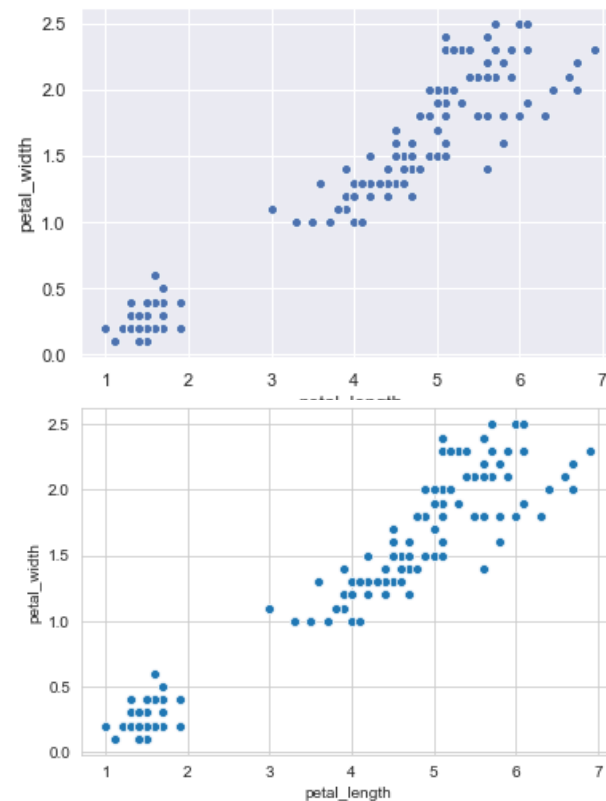
https://seaborn.pydata.org/generated/seaborn.axes_style.html#seaborn.axes_style

```
seaborn.set(context='notebook', style='darkgrid',
            palette='deep', font='sans-serif',
            font_scale=1, color_codes=True, rc=None)
```

style : dict, None, 또는 {darkgrid, whitegrid, dark, white, ticks}중 하나

```
1 sns.set(style="darkgrid")
2 sns.set_context("notebook", font_scale=1.5,
3                 rc={"lines.linewidth":2.5})
4 ax = sns.scatterplot(x="petal_length", y="petal_width",
5                     data=iris)
```

```
1 sns.set_style("whitegrid")
2 sns.set_context("notebook", font_scale=1.5,
3                 rc={"lines.linewidth":2.5})
4 ax = sns.scatterplot(x="petal_length", y="petal_width",
5                     data=iris)
```



3) Context 함수

3절. Seaborn > 3.2. 미적 속성 설정하기

- `set_context()` : **플롯팅 컨텍스트 파라미터** 설정
- 레이블의 크기, 선 그리고 그래프의 다른 요소들에는 영향을 미치지만 전반적인 스타일에는 영향을 미치지 않음

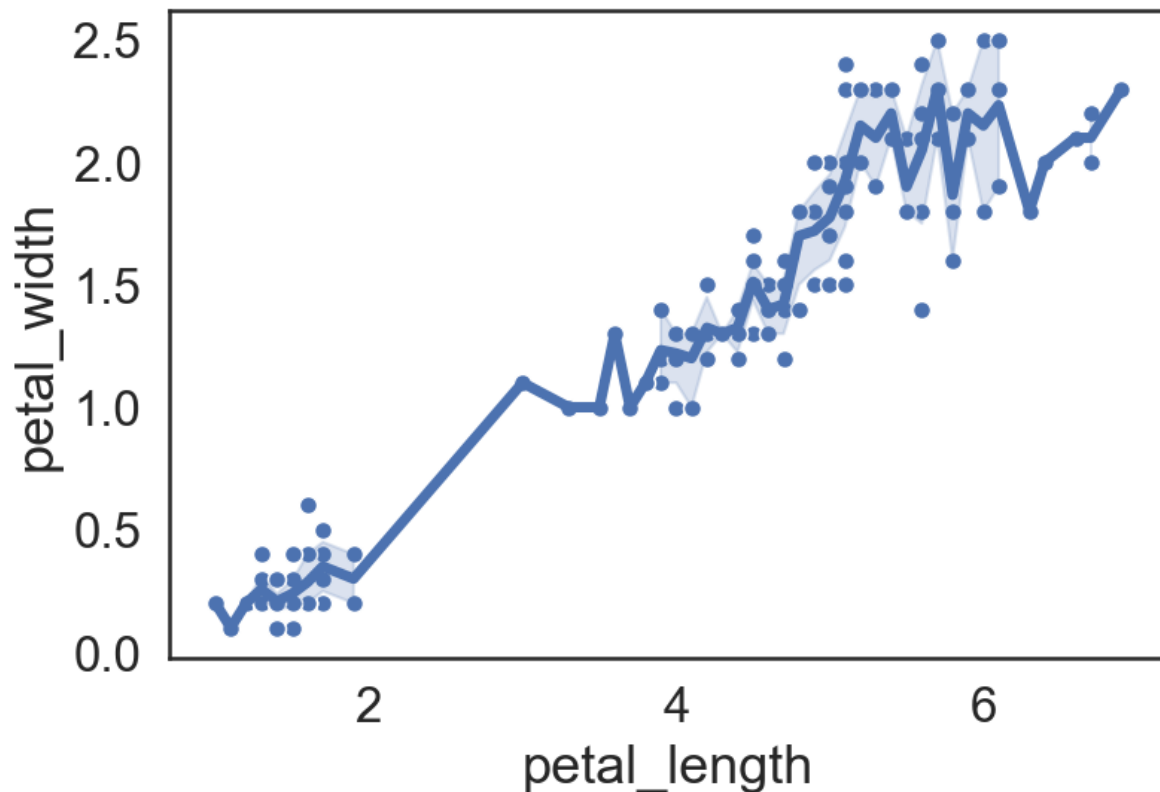
```
seaborn.set_context(context=None, font_scale=1, rc=None)
```

- 구문에서…
 - `context`: dict, None 또는 {paper, notebook, talk, poster} 중 하나를 지정
 - 기본 컨텍스트는 "notebook"이고 다른 컨텍스트는 "paper", "talk" 및 "poster"이며 각각 0.8, 1.3 및 1.6으로 배율이 조정
 - `font_scale`: float, 선택사항, 글꼴 요소의 크기를 독립적으로 조정하려면 배율 인수를 별도로 지정

3) 컨텍스트 지정

3절. Seaborn > 3.2. 미적 속성 설정하기

```
1 sns.set(style="white")
2 sns.set_context("notebook", font_scale=1.5, rc={"lines.linewidth":3.5})
3 ax = sns.scatterplot(x="petal_length", y="petal_width", data=iris)
4 ax = sns.lineplot(x="petal_length", y="petal_width", data=iris)
```



4) 컬러 팔레트

3절. Seaborn > 3.2. 미적 속성 설정하기

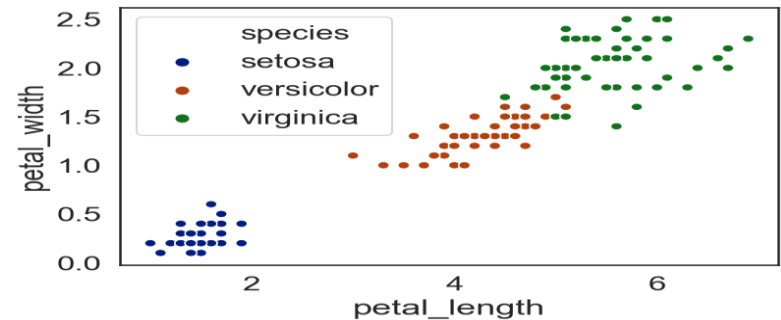
- `set_palette()` 함수는 컬러 팔레트를 지정

```
seaborn.set_palette(palette, n_colors=None)
```

- 구문에서...

- *palette*: 씨본의 컬러 팔레트(deep, muted, bright, pastel, dark, colorblind), Matplotlib의 컬러맵 그리고 hls, husl, 색상 리스트 등
- *n_colors*: 사이클의 색상 수. 기본 색상 수는 형식에 따라 다름

```
1 sns.set_palette("dark",3)
2 ax = sns.scatterplot(x="petal_length",
3                       y="petal_width",
4                       data=iris,
5                       hue="species")
```



https://seaborn.pydata.org/generated/seaborn.color_palette.html#seaborn.color_palette

<https://matplotlib.org/tutorials/colors/colormaps.html>

https://seaborn.pydata.org/tutorial/color_palettes.html

5) 스타일과 팔레트 함수들

3절. Seaborn > 3.2. 미적 속성 설정하기

함수	설명
<code>set([context, style, palette, font, ...])</code>	미적 매개 변수를 설정합니다.
<code>axes_style([style, rc])</code>	플롯의 미적 스타일에 대한 매개 변수를 반환합니다.
<code>set_style([style, rc])</code>	플롯의 미적 스타일을 설정합니다.
<code>plotting_context([context, font_scale, rc])</code>	Figure의 요소를 크기 조절하기 위해 매개 변수를 반환합니다.
<code>set_context([context, font_scale, rc])</code>	플롯 컨텍스트 매개 변수를 설정합니다.
<code>set_color_codes([palette])</code>	matplotlib의 색 단축이 어떻게 해석될지를 변경합니다.
<code>reset_defaults()</code>	모든 RC 매개 변수를 기본 설정으로 복원합니다.
<code>reset_orig()</code>	모든 RC 매개 변수를 원래 설정으로 복원합니다.

함수	설명
<code>set_palette(palette[, n_colors, desat, ...])</code>	seaborn 팔레트를 사용하여 matplotlib 색상주기를 설정합니다.
<code>color_palette([palette, n_colors, desat])</code>	색상 표를 정의하는 색상 목록을 반환합니다.
<code>husl_palette([n_colors, h, s, l])</code>	HUSL 색상 공간에서 균일 한 간격으로 색상 세트를 가져옵니다.
<code>hls_palette([n_colors, h, l, s])</code>	HLS 색상 공간에서 균일하게 간격을 둔 색상 세트를 가져옵니다.
<code>cubehelix_palette([n_colors, start, rot, ...])</code>	Cubehelix 시스템에서 순차 팔레트를 만듭니다.
<code>dark_palette(color[, n_colors, reverse, ...])</code>	어두운 곳에서 밝은 곳까지 블렌드 되는 순차 팔레트를 만듭니다.
<code>light_palette(color[, n_colors, reverse, ...])</code>	빛에서 색상으로 블렌드 되는 순차 팔레트를 만듭니다.
<code>diverging_palette(h_neg, h_pos[, s, l, sep, ...])</code>	두 개의 HUSL 색상 사이에 분기 팔레트를 만듭니다.
<code>blend_palette(colors[, n_colors, as_cmap, input])</code>	색상 목록 사이에 블렌드 되는 팔레트를 만듭니다.
<code>xkcd_palette(colors)</code>	xkcd 색상에서 색상 이름으로 팔레트를 만듭니다.
<code>crayon_palette(colors)</code>	Crayola 크레용에서 색상 이름을 가진 팔레트를 만듭니다.
<code>mpl_palette(name[, n_colors])</code>	Matplotlib 팔레트에서 개별 색상을 반환합니다.

3.3. Seaborn으로 그래프 그리기

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기

- 관계형(Relational), 범주형(Categorical), 분포(Distribution), 회귀(Regression), 행렬(Matrix)과 관련된 그래프 함수들 제공
- 이들 함수는 스스로 그래프를 그리는 기능이 있기 때문에 그래프 함수를 실행하면 바로 그래프를 그릴 수 있음
- 이들 그래프 함수들은 그래프가 그려지는 Axes(Matplotlib의 AxesSubplot) 객체를 반환하기 때문에 서브플롯(subplots())으로 나눈 축(axes) 영역에 그래프를 그릴 수 있음
- 씨본은 하나의 화면에 여러 개 그래프를 그릴 수 있도록 그리드 객체(FacetGrid, PairGrid, JointGrid)를 지원하기 때문에 figure를 데이터에 따라 축의 수를 나눠서 그래프를 그릴 수 있음

1) Relational plots : 관계형 그래프

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기

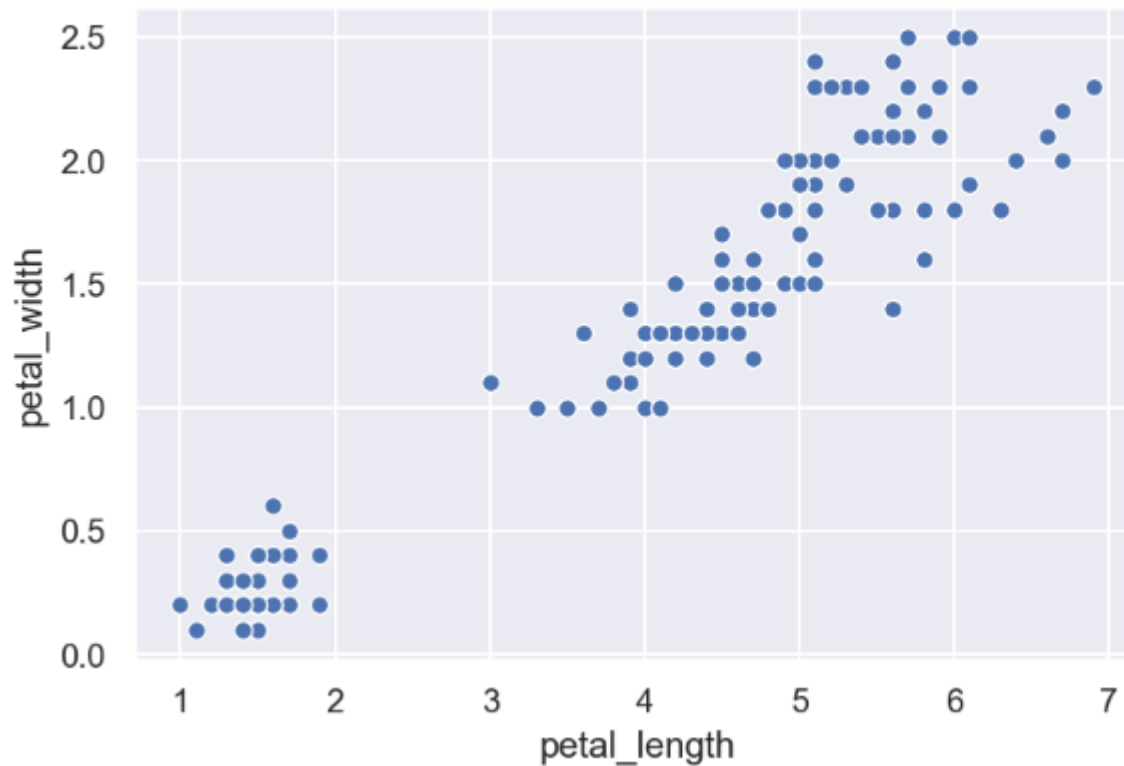
함수	설명
<code>relplot([x,y, hue, size, style, data, row, ...])</code>	관계형 플롯을 FacetGrid에 그리기위한 Figure 레벨 인터페이스입니다.
<code>scatterplot([x, y, hue, style, size, data, ...])</code>	몇 가지 의미 있는 그룹화가 가능한 산점도(scatter) 그래프를 그립니다.
<code>lineplot([x, y, hue, size, style, data, ...])</code>	몇 가지 의미 그룹화가 가능한 선(line) 그래프를 그립니다.

scatterplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

- scatterplot()은 산점도 그래프를 그려줌

```
1 ax = sns.scatterplot(x="petal_length", y="petal_width", data=iris)
```

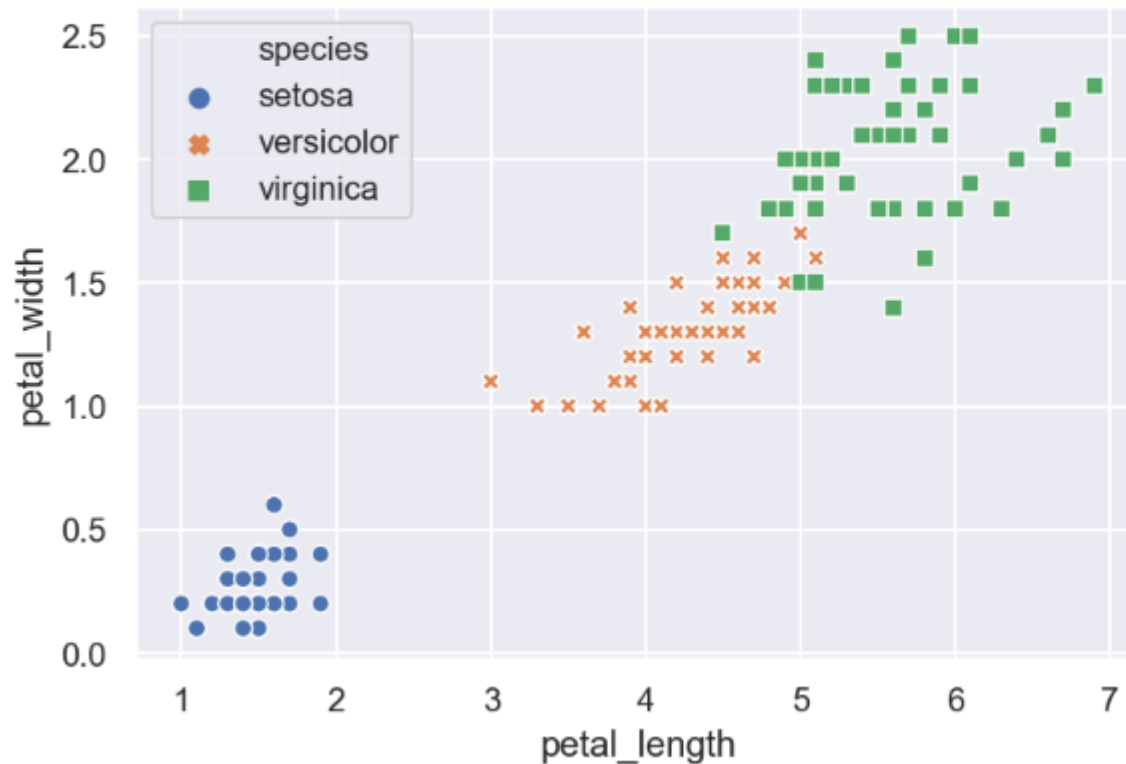


scatterplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

● 다른 변수로 그룹화 하고 다른 색상과 점의 스타일로 그룹을 표시

```
1 ax = sns.scatterplot(x="petal_length", y="petal_width",
2                       hue="species", style="species", data=iris)
```

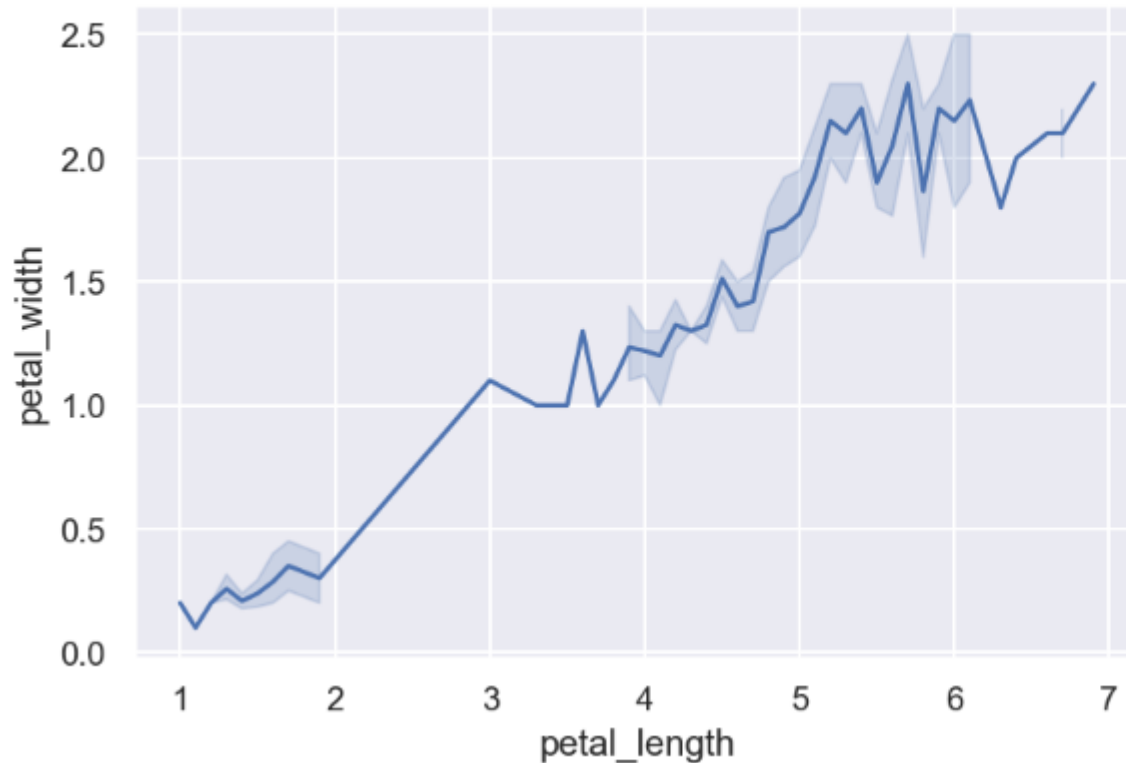


lineplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

- 신뢰 구간을 나타내는 오차 밴드가 있는 단일 선 그래프

```
1 ax = sns.lineplot(x="petal_length", y="petal_width", data=iris)
```

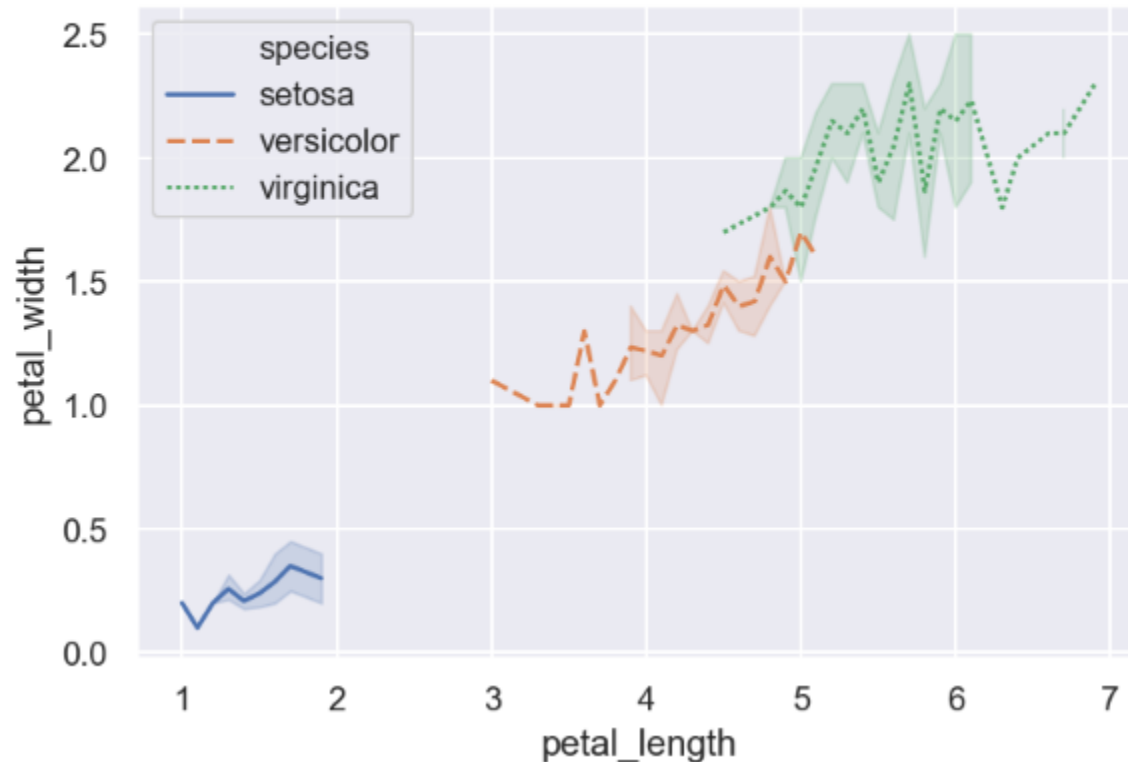


lineplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

● 변수로 그룹화 하고 그룹별로 다른 색상으로 그룹을 표시

```
1 ax = sns.lineplot(x="petal_length", y="petal_width",
2                   hue="species", style="species", data=iris)
```

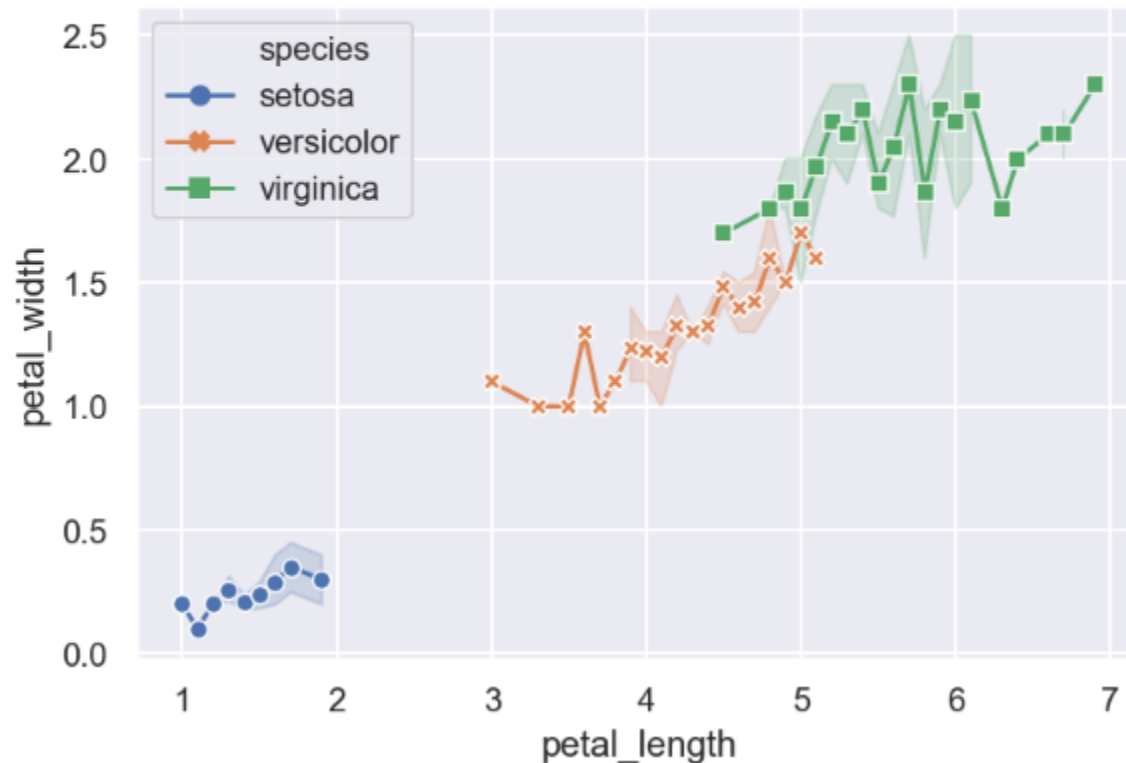


lineplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

● 점의 스타일로 그룹을 표시하고 선의 스타일은 모두 동일하게 설정

```
1 ax = sns.lineplot(x="petal_length", y="petal_width",
2                   hue="species", style="species",
3                   markers=True, dashes=False, data=iris)
```

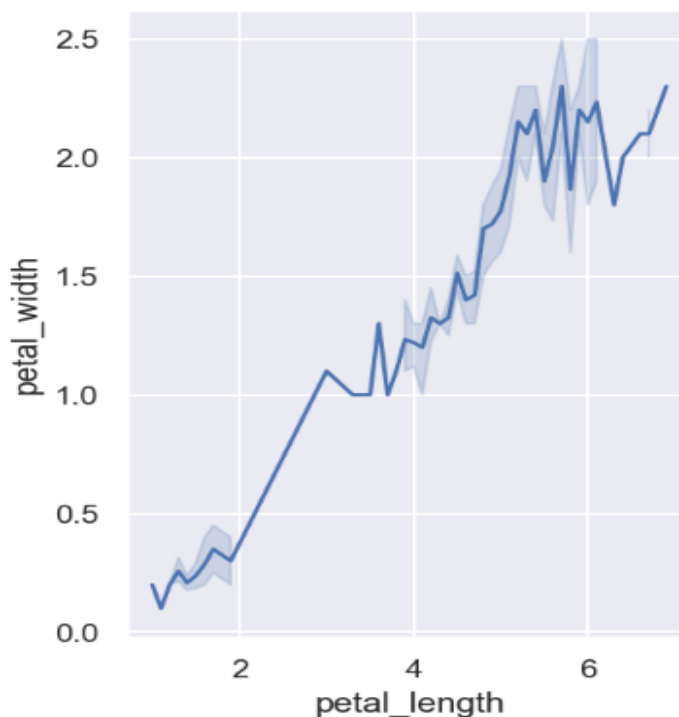
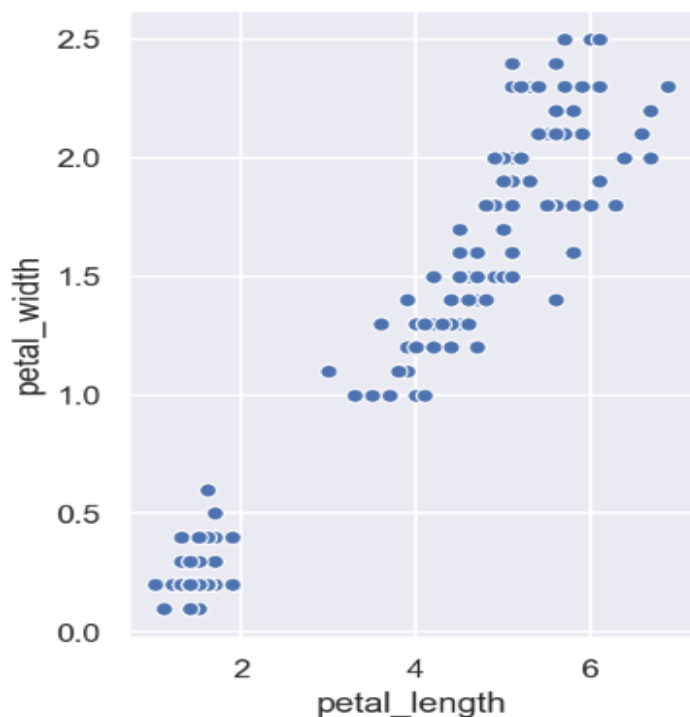


축에 그리기

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

- `pyplot.subplots()` 함수로 그래프 영역을 나눈 후 각 영역에 플롯팅

```
1 fig, axes = plt.subplots(ncols=2, figsize=(8,5))
2 plt.subplots_adjust(wspace=0.3)
3 sns.scatterplot(x="petal_length", y="petal_width", data=iris, ax=axes[0])
4 sns.lineplot(x="petal_length", y="petal_width", data=iris, ax=axes[1])
5 plt.show()
```

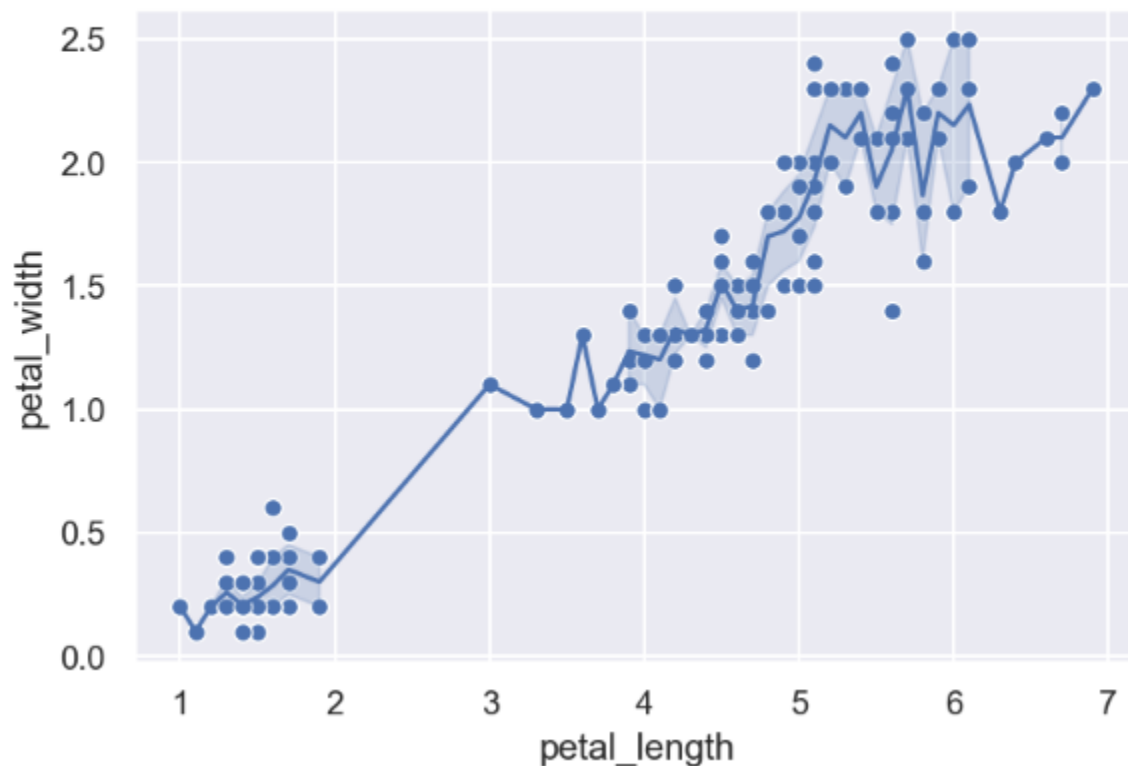


그래프 겹쳐 그리기

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 1) Relational plots : 관계형 그래프

● 축 하나에 그래프를 여러 개 그릴 수 있음

```
1 ax = sns.scatterplot(x="petal_length", y="petal_width", data=iris)
2 ax = sns.lineplot(x="petal_length", y="petal_width", data=iris)
```



2) Categorical plots : 범주형 그래프

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기

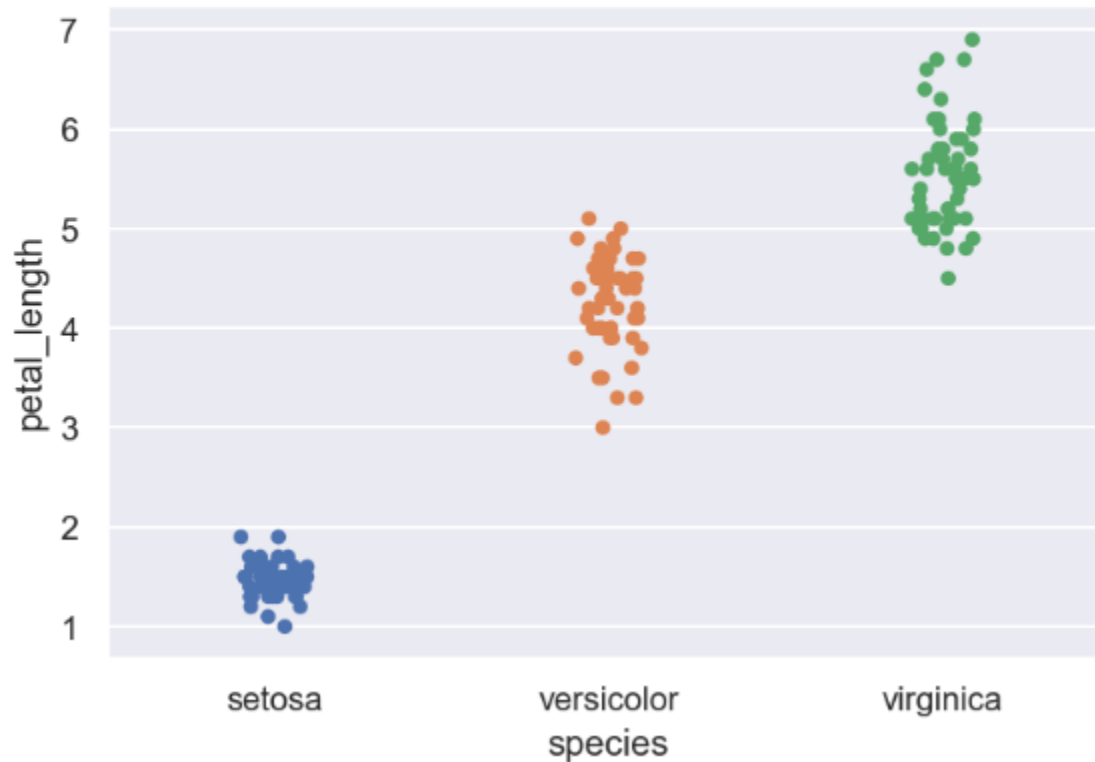
함수	설명
<code>catplot([x, y, hue, data, row, col, ...])</code>	범주 형 플롯을 FacetGrid에 그리기위한 Figure 레벨의 인터페이스입니다.
<code>stripplot([x, y, hue, data, order, ...])</code>	하나의 변수가 범주형인 산점도를 그립니다.
<code>swarmplot([x, y, hue, data, order, ...])</code>	중첩되지 않는 점들로 범주형 산점도를 그립니다.
<code>boxplot([x, y, hue, data, order, hue_order, ...])</code>	범주와 관련된 분포를 표시하는 박스 플롯을 그립니다.
<code>violinplot([x, y, hue, data, order, ...])</code>	박스 플롯과 커널 밀도 추정치의 조합을 그립니다.
<code>boxenplot([x, y, hue, data, order, ...])</code>	더 큰 데이터 세트를 위한 향상된 상자 플롯을 그립니다.
<code>pointplot([x, y, hue, data, order, ...])</code>	산점도 그림문자(glyph)를 사용하여 점 추정치 및 신뢰 구간을 표시합니다.
<code>barplot([x, y, hue, data, order, hue_order, ...])</code>	포인트 추정치와 신뢰 구간을 직사각형 막대로 표시합니다.
<code>countplot([x, y, hue, data, order, ...])</code>	막대를 사용하여 각 범주 구간의 관측 수를 표시합니다.

stripplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

● 변수 하나가 범주형인 산점도

```
1 iris = sns.load_dataset("iris")  
2 ax = sns.stripplot(x="species", y="petal_length", data=iris)
```

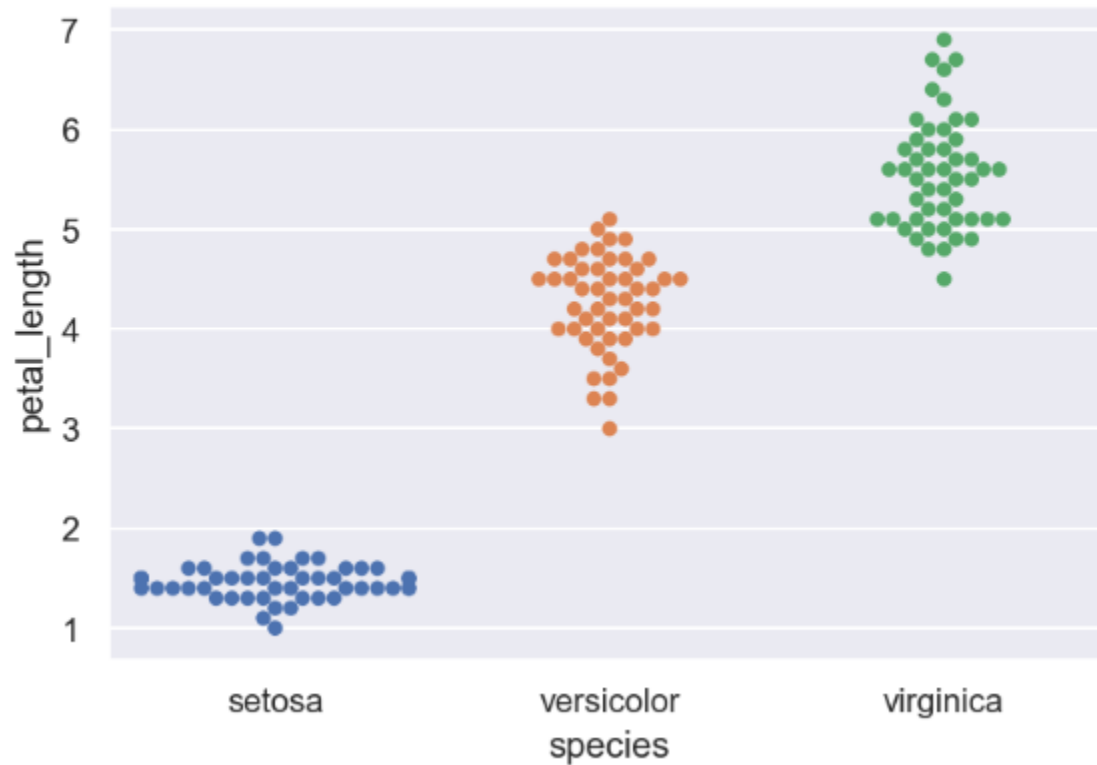


swarmplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

- 범주형 산점도의 점들이 중첩되지 않게 플롯팅

```
1 ax = sns.swarmplot(x="species", y="petal_length", data=iris)
```

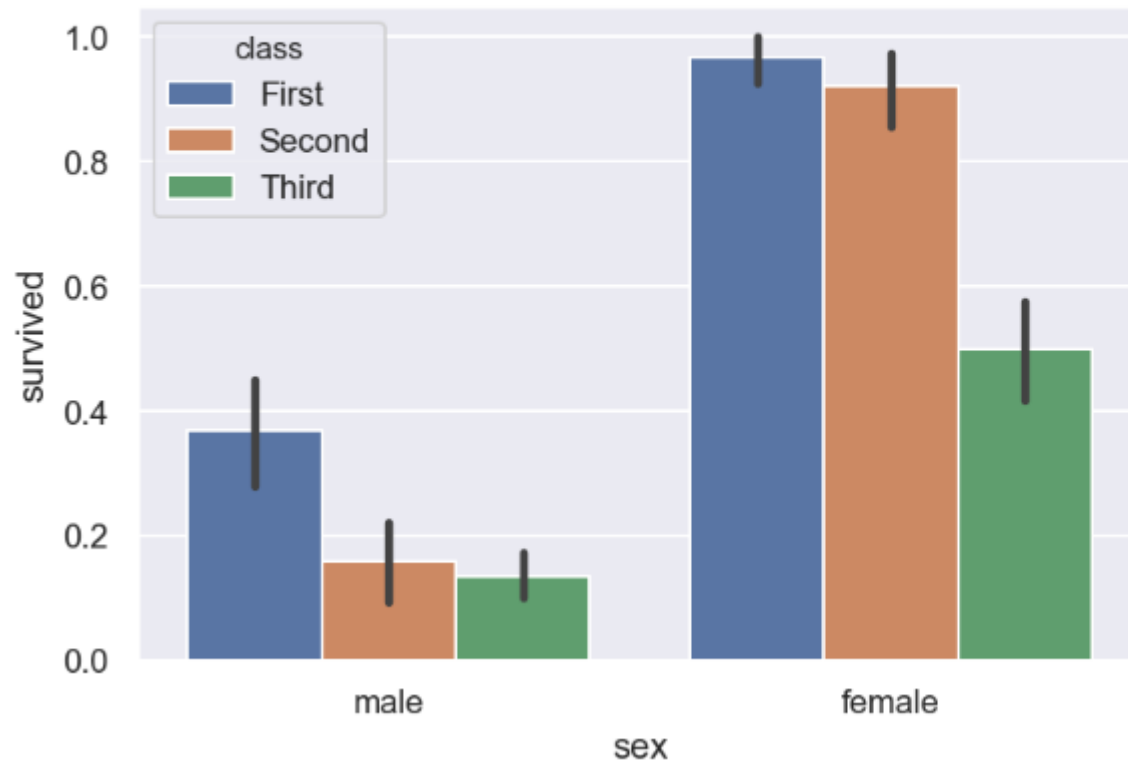


barplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

● 포인트 추정치와 신뢰 구간을 막대그래프로 표시

```
1 titanic = sns.load_dataset("titanic")
2 ax = sns.barplot(x="sex", y="survived", hue="class", data=titanic)
```



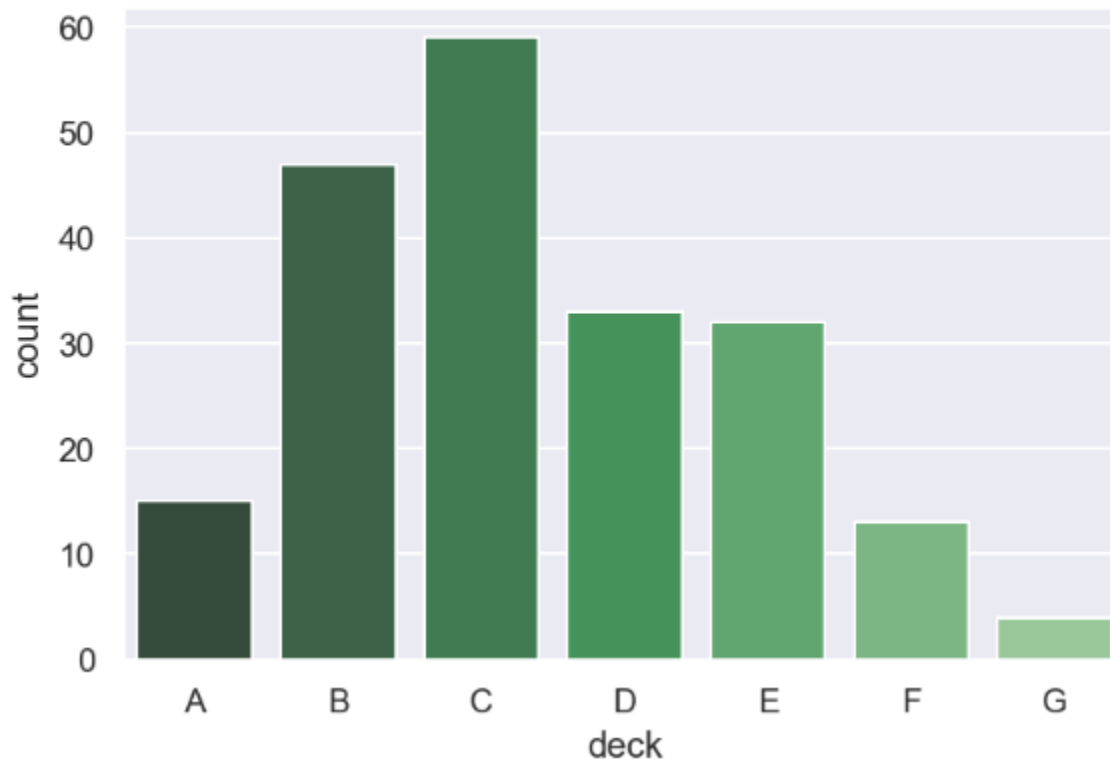
막대그래프는 표현 값에 비례하여
높이와 길이를 가진 직사각형 막
대로 범주형 데이터를 표현

countplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

- 막대를 사용하여 각 범주 구간의 관측 수를 표시

```
1 ax = sns.countplot(x="deck", data=titanic, palette="Greens_d")
```

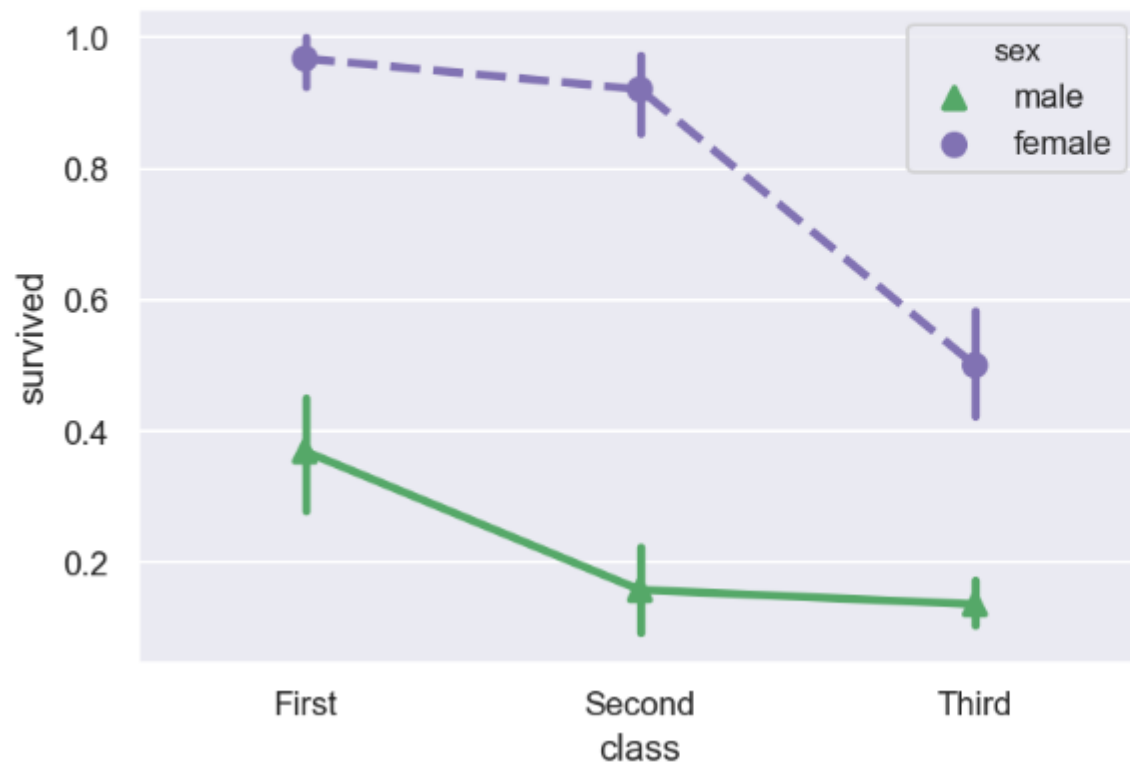


pointplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

● 산점도 그림문자를 사용하여 점 추정치 및 신뢰 구간을 표시

```
1 ax = sns.pointplot(x="class", y="survived", hue="sex", data=titanic,
2                     palette={"male": "g", "female": "m"},
3                     markers=["^", "o"], linestyle=["-", "--"])
```

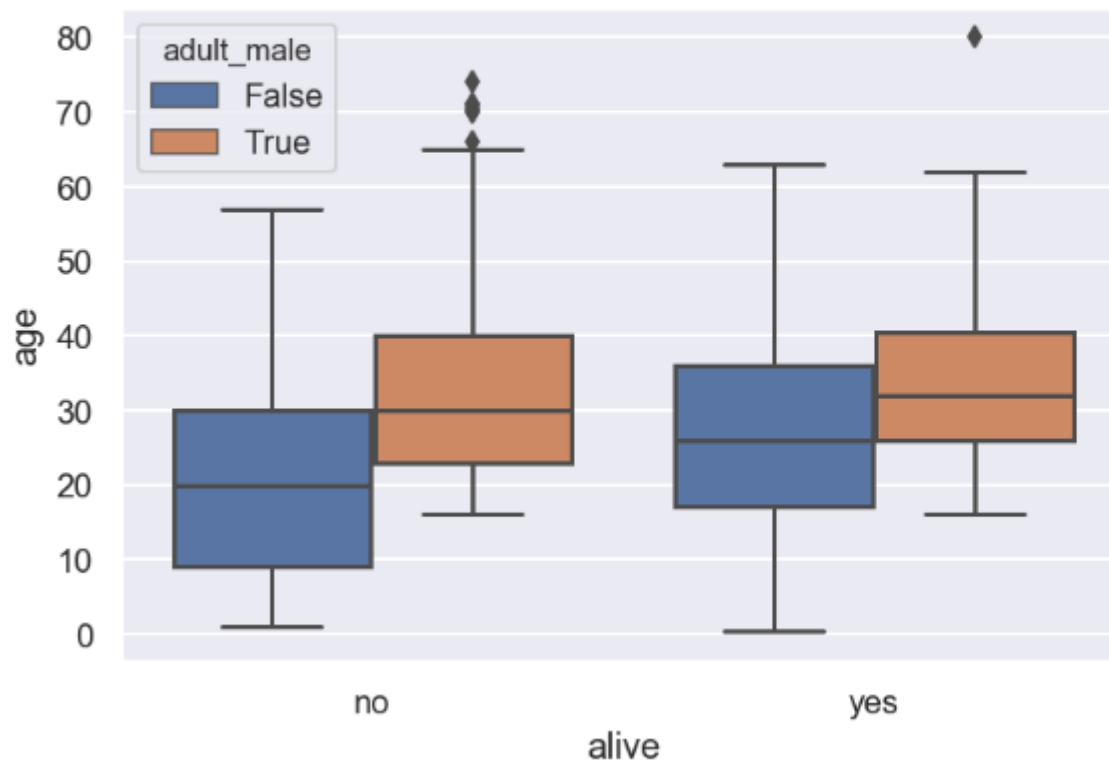


boxplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

● 사분위 그래프

```
1 ax = sns.boxplot(x="alive", y="age", hue="adult_male", data=titanic)
```



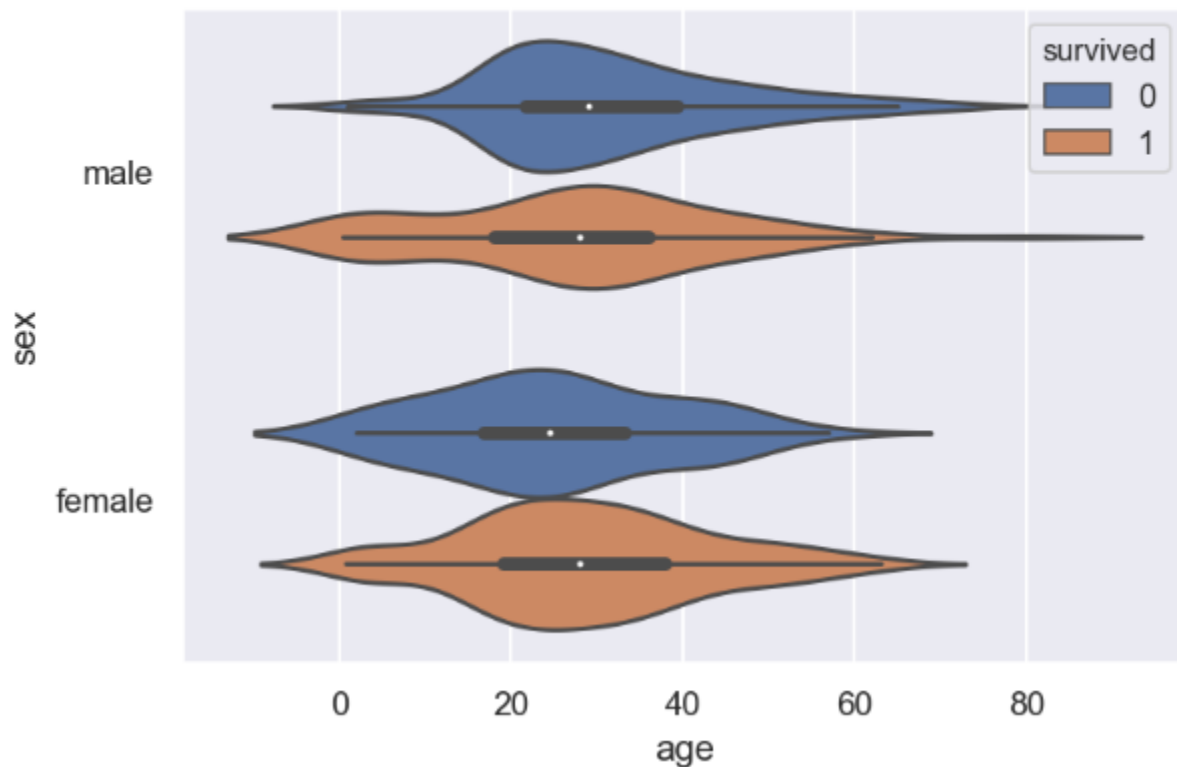
violinplot

<http://autodeskresearch.com/publications/samestats>

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

- 박스 플롯과 커널 밀도 추정치(KDE; Kernel Density Estimation)의 조합을 그림

```
1 ax = sns.violinplot(x="age", y="sex", hue="survived", data=titanic)
```

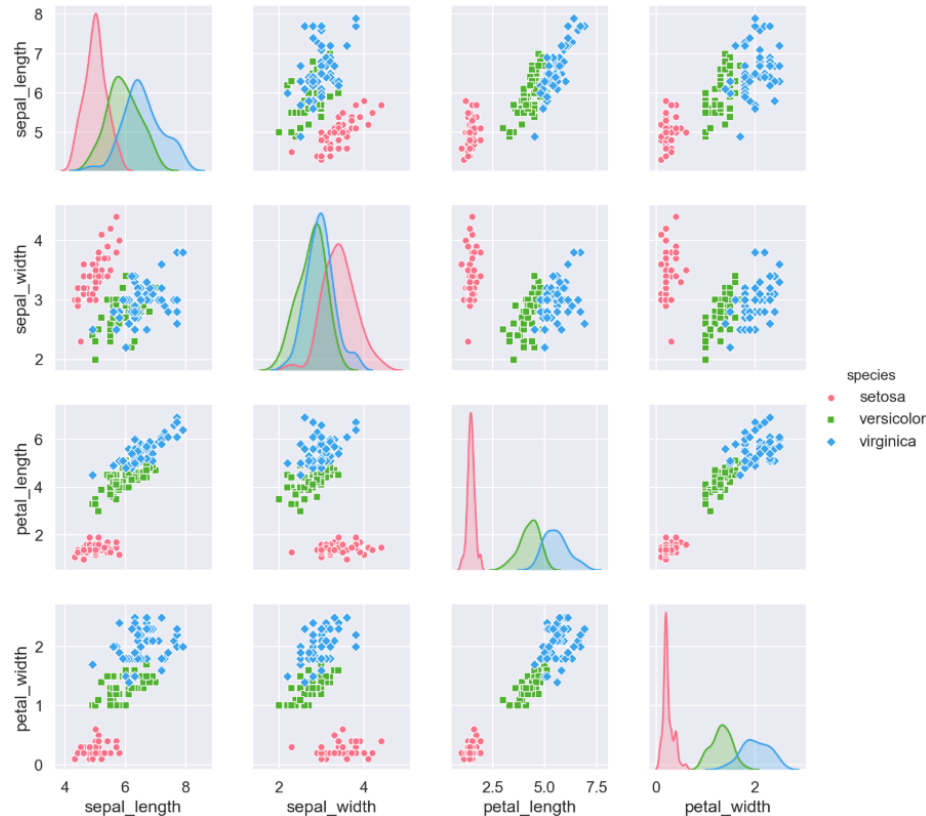


pairplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 3) Distribution plots : 분포형 그래프

● 페어 플롯(쌍 관계 그래프)을 그림

```
1 sns.set(font_scale=1.2)
2 ax = sns.pairplot(iris, hue="species", palette="husl", markers=["o", "s", "D"])
```



4) Regression plots : 회귀 그래프

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기

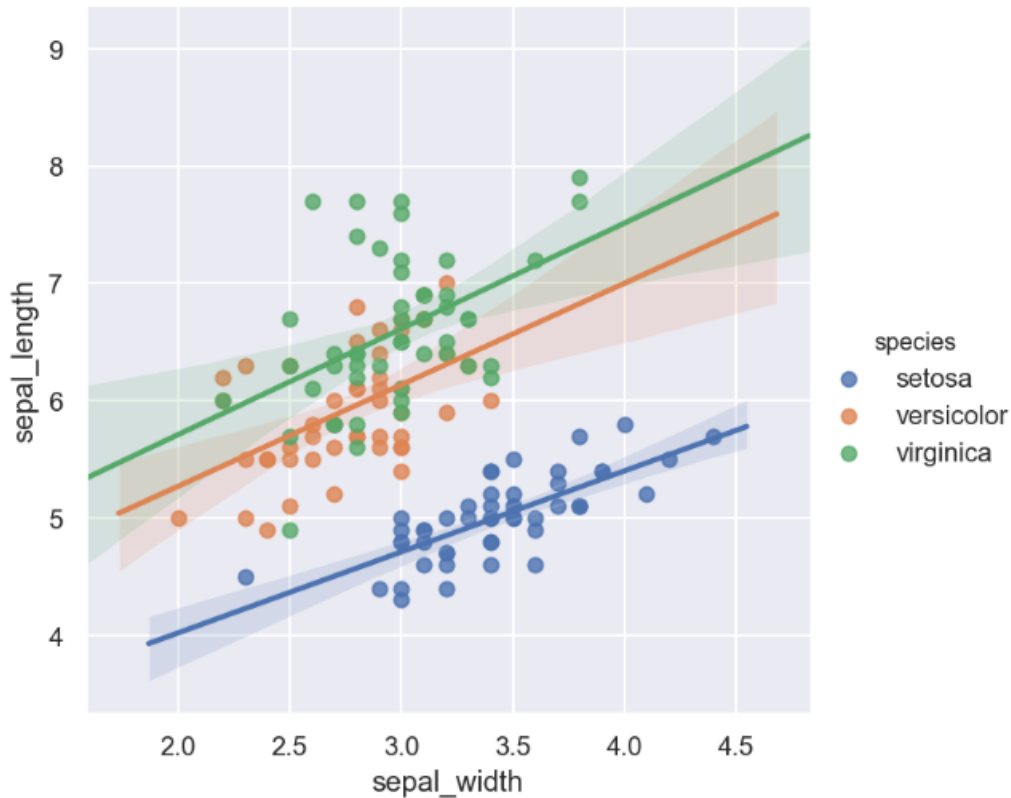
함수	설명
<code>lplot(x, y, data[, hue, col, row, palette, ...])</code>	데이터와 회귀 모델 적합을 FacetGrid에 그립니다.
<code>regplot(x, y[, data, x_estimator, x_bins, ...])</code>	데이터와 선형 회귀 모델을 그립니다.

Implot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 4) Regression plots : 회귀 그래프

● 데이터를 이용해 선형 회귀 모델을 만들고 그래프로 표현

```
1 sns.set()  
2 ax = sns.lmplot(x="sepal_width", y="sepal_length", data=iris,  
3                 hue="species")
```

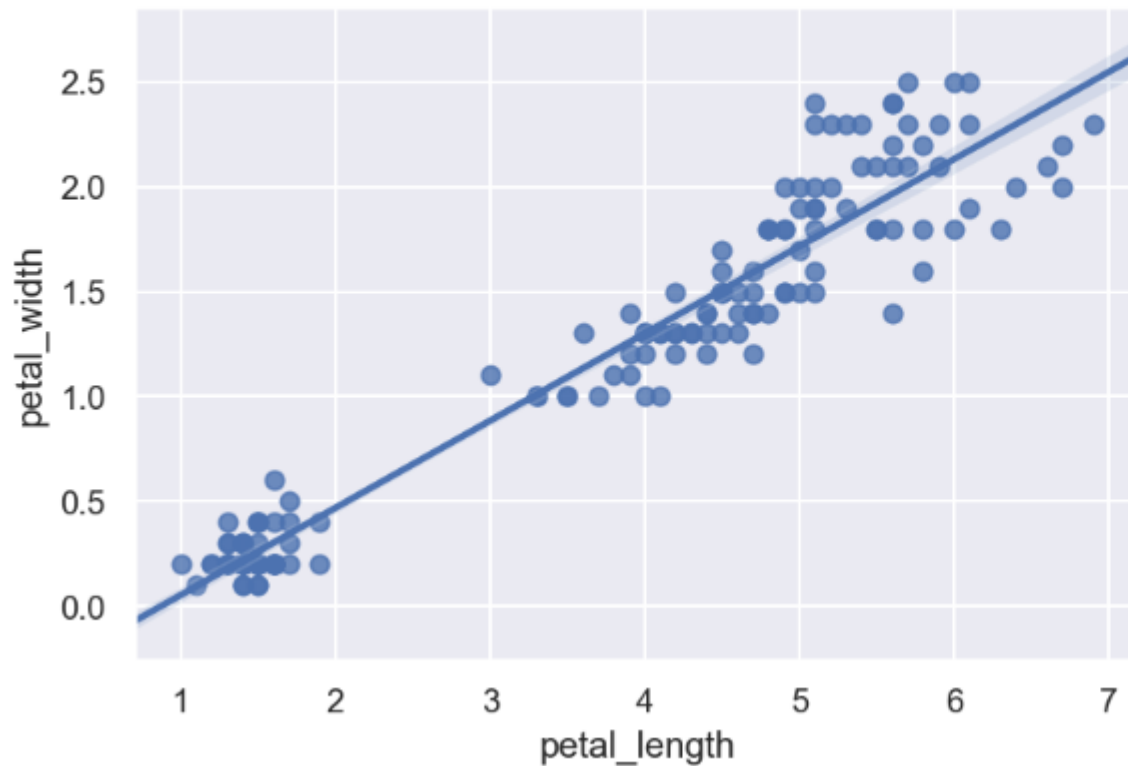


regplot

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 4) Regression plots : 회귀 그래프

- 두 변수를 이용해서 선형 회귀 모형을 만들고 그래프로 표현

```
1 ax = sns.regplot(x="petal_length", y="petal_width", data=iris)
```



❖ 이 함수가 회귀 결과를 그래프로 그려주지만 이것을 이용해서 회귀의 수치 결과를 찾거나 모델의 결정계수 (R2) 등을 출력할 수 있는 방법은 없습니다.

5) Matrix plots : 행렬 그래프

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기

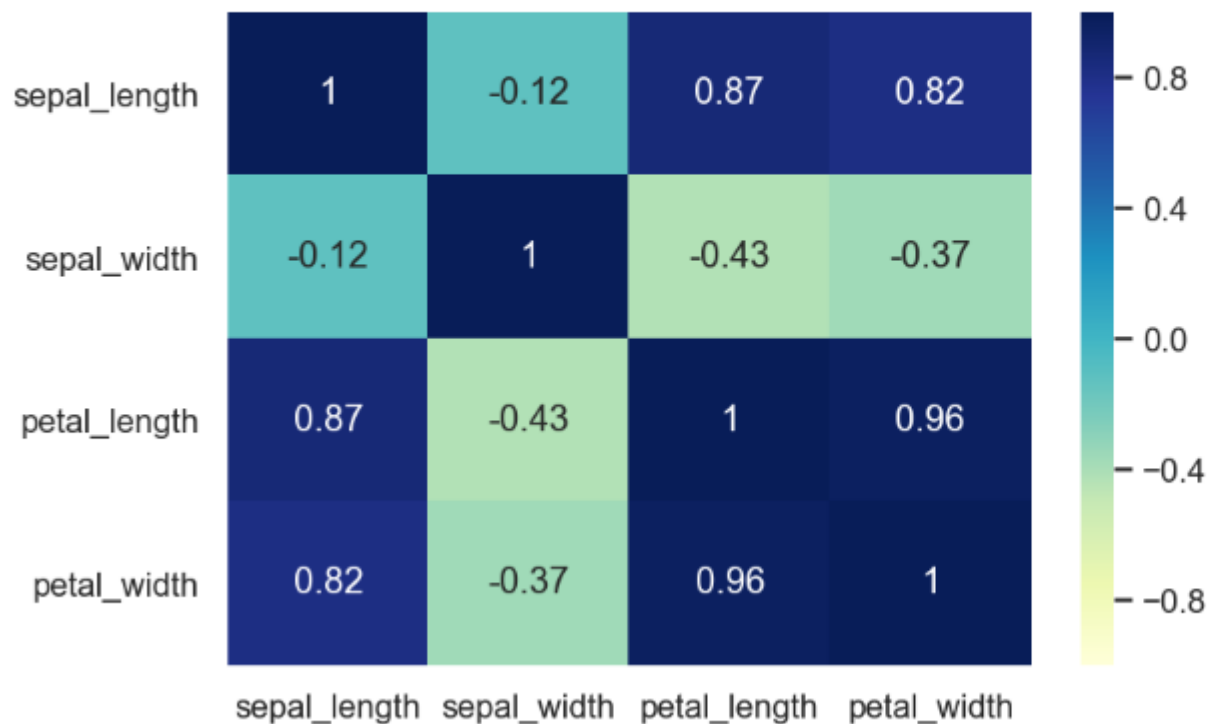
함수	설명
<code>heatmap(data[, vmin, vmax, cmap, center, ...])</code>	데이터를 색으로 인코딩 된 직사각형 행렬(히트맵)로 표시합니다.

heatmap

3절. Seaborn > 3.3. Seaborn으로 그래프 그리기 > 5) Matrix plots : 행렬 그래프

● 데이터를 색으로 인코딩 된 직사각형 행렬(히트맵)로 표시

```
1 ax = sns.heatmap(iris.corr(), vmin=-1, vmax=1, annot=True, cmap="YlGnBu")
```



annot=True이면 값을 표시

1) FacetGrid

3절. Seaborn > 3.4. Multi-plot grids

● 쌍 관계를 그리기 위한 다중 플롯 그리드 생성

함수	설명
<code>FacetGrid(data[, row, col, hue, col_wrap, ...])</code>	조건부 관계를 그리기 위한 다중 플롯 그리드를 표시합니다.
<code>FacetGrid.map(func, *args, **kwargs)</code>	각 패킷의 데이터 하위 집합에 플로팅 기능을 적용합니다.
<code>FacetGrid.map_dataframe(func, *args, **kwargs)</code>	.map과 비슷하지만 args를 문자열로 전달하고 kwargs에 데이터를 삽입합니다.

https://seaborn.pydata.org/tutorial/axis_grids.html#conditional-small-multiples

map

3절. Seaborn > 3.4. Multi-plot grids

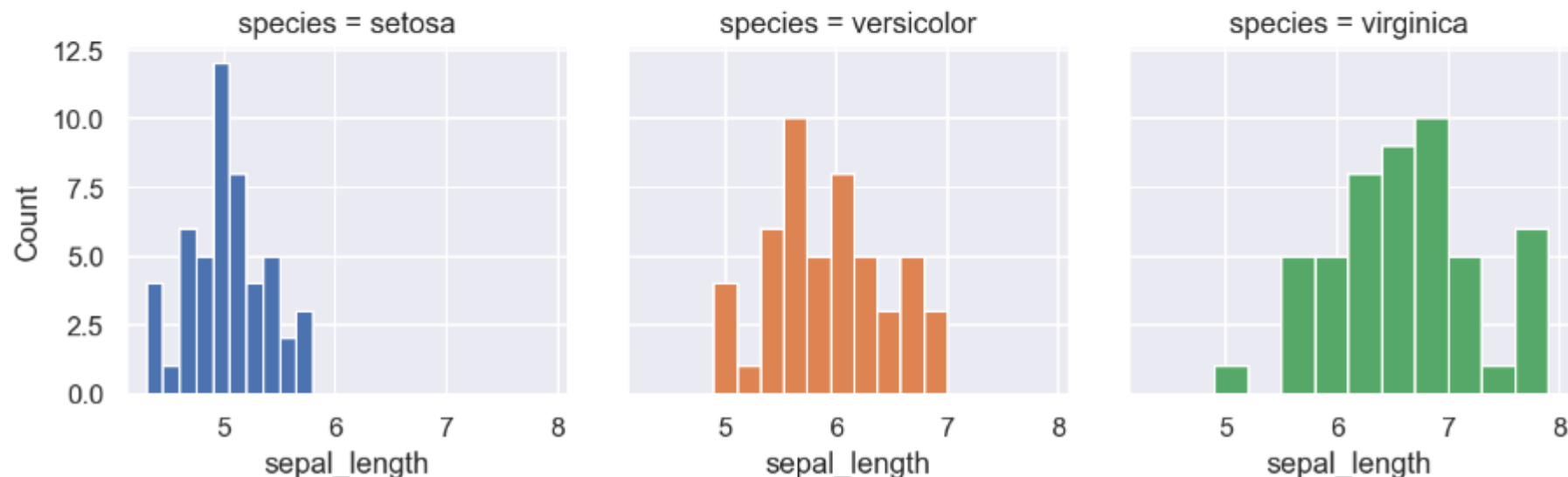
```

1 import seaborn as sns
2 sns.set()
3 iris = sns.load_dataset("iris")
4
5 g = sns.FacetGrid(iris, col="species", hue="species")
6 g.map(plt.hist, "sepal_length")
7 g.set_axis_labels("sepal_length", "Count")

```

- ❖ 해당 영역에 그래프를 그리기 위해 map() 메소드 이용
- ❖ 각각의 영역에 func 매개변수에 지정하는 함수를 이용해서 그래프를 그림

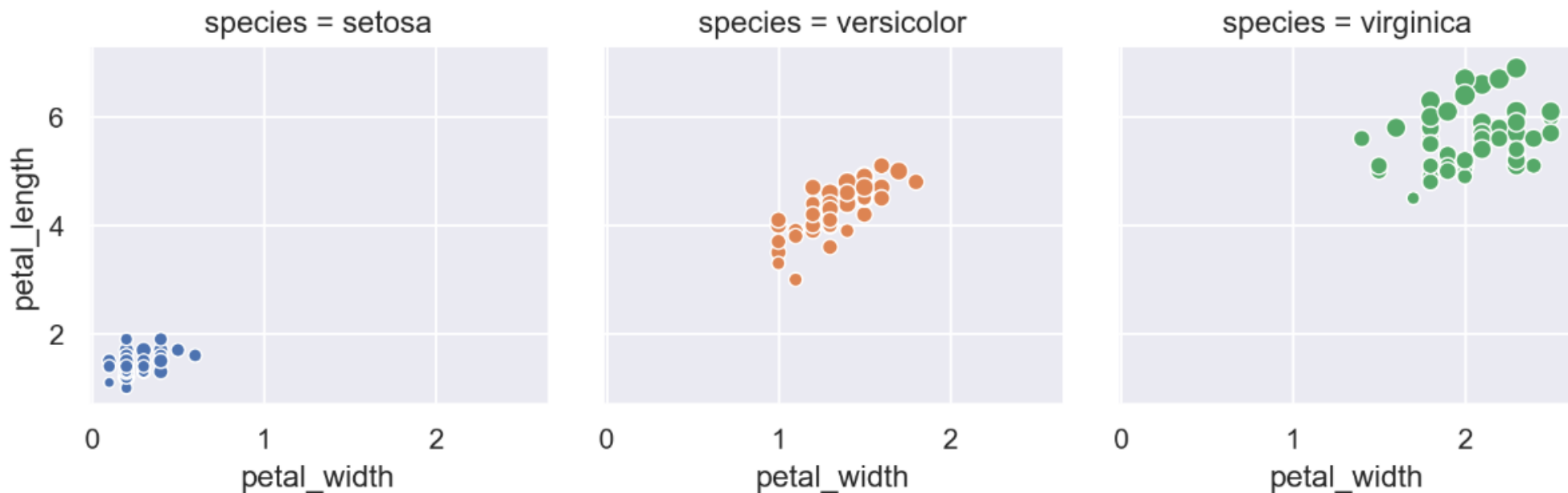
<seaborn.axisgrid.FacetGrid at 0x1f0f8088588>



map

3절. Seaborn > 3.4. Multi-plot grids

```
1 g = sns.FacetGrid(iris, col="species", hue="species")  
2 g.map(sns.scatterplot, 'petal_width', 'petal_length',  
3       size=iris.sepal_length)
```



map

3절. Seaborn > 3.4. Multi-plot grids

```
1 import seaborn as sns
2 titanic = sns.load_dataset("titanic")
3
4 g = sns.FacetGrid(titanic, col="survived", row="sex")
5 g.map(plt.hist, "age")
```

